



University of Bahrain
College of Information Technology
Department of Information Systems
B.Sc. In Cybersecurity

SECURE NATURAL LANGUAGE TO BASH EXECUTION - SHELLSENTRY

Prepared by

Aljazi Ali Almujaaddam 202202227

Dana Hussain Alhayki 202203431

For

ITCY 499

Senior Project

Academic Year 2025-2026-Semester 2

Project Supervisor:
Dr. Abdulla Khalifa Aldoseri

12 May 2026

Abstract

A secure, web-based prototype that takes admin requests in natural language to a validated bash command to execute on a remote Linux system. The project tackles the challenges of manual shell management and the execution of commands generated by AI, using a combination of authenticated access, retrieval-augmented generation (RAG), host-aware context collection, deterministic command validation, and SSH-based multi-server orchestration. Flask, Python, Paramiko, FAISS, and an OpenAI-compatible Large Language Model were used to implement the system. Functional, security, and performance testing are done in a controlled environment in the virtual lab. The results demonstrated successful authentication of execution, multi-host command orchestration, blocking of unsafe commands, and structured audit logging with a reliable report. ShellSentry showcases how AI-driven shell automation can become much more secure by implementing multiple-level validation, execution policies, and accountability.

Acknowledgments

(وَأَخِرُ دَعْوَاهُمْ أَنْ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ)
اللهم لك الحمد حتى ترضى ولك الحمد إذا رضيت ولك الحمد بعد الرضى.

I would like to thank my project supervisor for their guidance, constant support, and feedback during the creation of this senior project. They have given generous advice and encouragement, which was fundamental to both the technical and academic development of this work.

A special thanks to all the faculty members of the department who were able to provide the knowledge and environment to complete this project.

I would like to express my appreciation to my family for being patient, encouraging, and supportive through this process. Their motivation and understanding helped me to overcome the challenges during the development of this project.

Last but not least, I need to thank you all who helped ShellSentry into existence, either through direct or indirect contributions.

Table of Contents

ABSTRACT.....	II
ACKNOWLEDGMENTS	III
LIST OF TABLES	VI
LIST OF FIGURES	VII
CHAPTER 1 INTRODUCTION	1
1.1 PROBLEM STATEMENT	1
1.2 PROJECT OBJECTIVES	1
1.2.1 Scope	1
1.2.2 Target Audience	2
1.2.3 Impact	2
1.2.4 Limitations	2
1.2.5 Assumptions	2
1.3 RELEVANCE/SIGNIFICANCE OF THE PROJECT	2
1.4 REPORT OUTLINE	3
CHAPTER 2 LITERATURE REVIEW	4
2.1 Background	4
2.2 Related Work	4
2.2.1 ScriptSmith — Automated Bash Script Generation, Assessment, and Refinement	4
2.2.2 Leveraging machine learning for natural language to bash translation (LLM-Supported NATURE)	5
2.2.3 LLM-as-a-Judge for Automatic Code Validation and Refinement	5
2.3 Summary	6
2.4 Relevance to Our Project	6
CHAPTER 3 PROJECT MANAGEMENT	7
3.1 PROCESS MODEL	7
3.2 RISK MANAGEMENT	8
3.3 PROJECT ACTIVITIES PLAN	9
3.4 CHAPTER SUMMARY	10
CHAPTER 4 REQUIREMENT COLLECTION AND ANALYSIS	11
4.1 REQUIREMENT ELICITATION	11
4.2 SYSTEM REQUIREMENTS	13
4.2.1 Functional Requirements	13
4.2.2 Non-functional requirements	14
4.3 SYSTEM MODELS	15
4.3.1 System architecture	15
4.3.2 Entity relationship diagram (ERD)	16
4.4 CHAPTER SUMMARY	16
CHAPTER 5 SYSTEM DESIGN	17
5.1 INTRODUCTION	17
5.2 DATABASE DESIGN	17
5.2.1 Purpose of the database in ShellSentry	17
5.2.2 Logical schema and data dictionary	18
5.2.3 Design decisions and scope boundaries	19
5.3 USER INTERFACE AND INTERACTION DESIGN	19
5.3.1 Landing, login, and registration	19
5.3.2 Main dashboard: request composition and targets	21
5.3.3 Results presentation	21

5.3.4 Error and Blocked-Command Feedback	23
5.3.5 Navigation and session flow.....	24
5.4 CORE ALGORITHMS.....	25
5.4.1 Natural-Language Execution Algorithm.....	25
5.4.2 Command Validation Algorithm.....	26
5.4.3 Parallel SSH Execution Algorithm	27
5.4.4 Script Archive and Safe Cron Algorithm	27
5.4.5 Host Context Collection Algorithm	28
5.4.6 RAG Retrieval Algorithm.....	28
5.5 MAPPING TO REQUIREMENTS	28
5.5.1 Functional Requirements Coverage.....	28
5.5.2 Non-Functional Requirements Coverage.....	29
5.6 CHAPTER SUMMARY	29
CHAPTER 6 SYSTEM IMPLEMENTATION AND TESTING.....	31
6.1 SYSTEM IMPLEMENTATION	31
6.1.1 Development environment and hardware specifications	31
6.1.2 Technology Stack Selection and Justification	31
6.1.3 Project structure and module organization	33
6.1.4 Configuration and deployment behavior	34
6.1.5 Implemented features vs Chapter 5 design.....	35
6.2 EVALUATION AND TESTING	35
6.2.1 Testing methodology.....	36
6.2.2 Functional testing.....	36
6.2.3 Security testing.....	40
6.2.4 Performance measurement.....	46
6.2.5 LLM comparison (Groq API models)	47
6.2.5.1 Comparison of Findings.....	48
6.2.5.2 Comparison Limitations	48
6.2.6 Comparison with a Real-World Solution	48
6.2.7 Strengths and Weaknesses.....	49
6.2.7.1 Strengths	50
6.2.7.2 Weaknesses.....	50
6.2.8 Discussion and Comparison with Literature	50
6.3 CHAPTER SUMMARY	50
CHAPTER 7 CONCLUSION AND FUTURE WORK.....	52
7.1 PROJECT LIMITATIONS	52
7.2 FUTURE WORK	52
7.3 CHAPTER SUMMARY	52
REFERENCES.....	54

List of Tables

Table 1: Identified risks and mitigation plans for ShellSentry.....	8
Table 2: ShellSentry project activity plan	9
Table 3: Requirement elicitation — questions and participant responses	11
Table 4: User entity — data dictionary.....	18
Table 5: ExecutionLog entity — data dictionary.....	18
Table 6: Main technologies and libraries used in ShellSentry.....	32
Table 7: Representative functional test scenarios.....	36
Table 8: Security-oriented checks	40
Table 9: Observed timing (replace with your measurements).....	47
Table 10: Two-model comparison under identical prompt/host conditions.....	47
Table 11: presents a comparison between ShellSentry and Shell-GPT.....	49

List of Figures

Figure 1: Phases of the waterfall model used for ShellSentry	7
Figure 2: ShellSentry project Gantt chart.....	10
Figure 3: System architecture.....	16
Figure 4: Entity relationship diagram (ERD)	16
Figure 5: Main Page User Interface	20
Figure 6: Log-in Page User Interface.....	20
Figure 7: Sign-up User Interface	20
Figure 8: Dashboard User Interface	21
Figure 9: Option Clickable Execution Command User Interface	21
Figure 10: Result User Interface	22
Figure 11: Explanation User Interface	22
Figure 12: Technical report User Interface.....	23
Figure 13: Error Handling.....	24
Figure 14: User flow.....	25
Figure 15: Project tree structure	33
Figure 16: F1: Unauthenticated access	38
Figure 17: F2: Login	38
Figure 18: F3: NL → Bash (read-only style query)	38
Figure 19: F4: Default targets.....	39
Figure 20: F5: Multi-host partial outcome	39
Figure 21: F6: Validation rejection	39
Figure 22: F7: Archive list	40
Figure 23: F8: Safe Cron list / schedule	40
Figure 24: F9: Safe Cron destructive block.....	40
Figure 25: S1 : Authenticated execution(P1).....	42
Figure 26: S1: Authenticated execution(P2).....	42
Figure 27: S2: Session-only dashboard	43
Figure 28: S3: SecurityLayer input rejection.....	43
Figure 29: S4: Post-LLM deterministic block.....	44
Figure 30: S5: Read-only enforcement (if READ_ONLY_EXECUTION=true).....	44
Figure 31: S6: Privileged execution gate (if ALLOW_ROOT_EXECUTION=false)	44
Figure 32: S7: Safe Cron anti-wipe	45
Figure 33: S8: Script archive re-validation	45
Figure 34: S9: XSS / injection in UI rendering	46
Figure 35: S10: Secret hygiene.....	46
Figure 36: LLM Versatile Configuration	48
Figure 37: LLM Instant Configuration	48

Chapter 1

Introduction

In this chapter, ShellSentry, a secure web-based tool for requesting a validation of a natural-language command and invoking it on remote Linux hosts, is introduced. This problem of safe, large-scale shell automation and the lack of adequate solutions are presented in Section 1.1. Section 1.2 provides an overview of the project goals, scope, audience, impact, and project constraints and assumptions. The contributions of this work are given in Section 1.3, and the outline of the rest of the report is given in Section 1.4.

1.1 Problem Statement

In the modern world, there are dozens or hundreds of Linux servers in use within an organization that need to be queried, monitored, and configured using bash commands. Typical tasks like listing active connections, checking disk space, sampling running services, backing up a database, and so on often involve an administrator typing the appropriate command, pointing to the appropriate host, and authenticating over SSH, and then repeating this on the entire fleet. The skill barrier also means that most analysts, auditors, and junior staff aren't accountable for the operations that they can't do, meaning that there's a real risk: If they mistype a flag on a production server, they can cause downtime or loss of data or open a security hole.

There is an appealing alternative, known as the Large Language Models (LLMs). SharePoint-GPT, GitHub Copilot in the bash shell, and Open Interpreter can transform natural language into bash (TheR1D, 2026), making the hurdle much lower. However, the risk of running LLM output on remote hosts is a new class of risk. Some of the most common ways that LLM-powered automation fails are through hallucinated destructive commands (`rm -rf /`, `dd if=...`) (OWASP Foundation, 2026), prompt-injection attacks that modify the system prompt, privilege abuse with sudo, pipe-to-shell exfiltration methods, and others .

There is today no widely used open framework that integrates a natural-language interface, several layers of validation of LLM outputs, host-aware prompting, retrieval grounding to a trusted knowledge base, and an audit trail of parallel execution over SSH. This is where ShellSentry comes in. Instead of relying on the LLM, ShellSentry assumes that all model outputs are untrusted and blocks them behind input rules, a curated retrieval layer, a whitelist/blacklist command validator, and a read-only execution policy.

1.2 Project Objectives

The purpose of this project is to create and deploy a secure, Web-based interface to transcribe natural language requests to validated Bash commands and run them securely on one or more remote Linux hosts.

1.2.1 Scope

The project results in a functional prototype that includes a Flask-based web interface (Flask Documentation, 2026), an authentication layer, an input-validation layer, a host-context probing mechanism over SSH, a Retrieval-Augmented Generation (RAG) layer based on a curated Bash knowledge base (Lewis, 2020), and an OpenAI-compatible LLM client. It will also contain a multi-rule command validator in a strict read-only mode, a parallel executor for SSH using Paramiko (Paramiko Documentation, 2026), an audit logging system, and a result-formatting layer including a secondary

LLM run to provide a plain-language explanation of the results of execution. Evaluation is conducted by a controlled set of natural language prompts that are run against virtual Linux hosts.

1.2.2 Target Audience

The system is designed for system administrators, security analysts, junior operations people, and IT auditors who need to gain quick and easy access to information from multiple Linux hosts without having to write Bash commands. It's additionally ideal for academic or educational settings, such as Linux administration, applied AI, and cybersecurity classes, where the tool offers the entire flow from natural language input to the proven generation of commands in a single response.

1.2.3 Impact

ShellSentry is designed in the context of the University of Bahrain, but many parts of the design apply to a wide range of contexts. The more targets the host fleet increases, the more effective it gets, because parallel SSH execution and host-aware prompting grow linearly with the number of targets, and the default read-only mode is intact. More enterprises are turning to agentic LLM workflows, which makes the project a valuable educational resource for the implementation of multi-layer defenses around an LLM in a security-conscious environment.

1.2.4 Limitations

This is the current prototype, which is intended mainly for read-only inspections and light scripting. Write operation is disabled by default through the use of the `READ_ONLY_EXECUTION` state change command; only use the command when `READ_ONLY_EXECUTION` is set to LOW, and it is not recommended outside controlled environments. The system is limited to the Linux hosts using only SSH, and Windows targets using WinRM are out of scope. Natural-language input is mainly supported in English, without any other language support for any of the available operations. The accuracy of generated commands also relies on the knowledge base and the type of model used to generate commands; smaller models tend to occasionally produce commands that are not accepted by the validator.

1.2.5 Assumptions

The design relies on the assumption of a modern Linux OS with the utilities of bash, ss, systemctl, and coreutils being available on target hosts. It also makes certain assumptions about how SSH access is configured (via SSH keys or per-server credentials stored in a .env file outside the source tree) and that the Flask service works in a controlled and authenticated environment. The LLM endpoint must be an OpenAI-compatible API like OpenAI, Groq (OpenAI, 2026), or a local Ollama gateway. Further, it is assumed that the RAG knowledge base has only trusted manually written command examples.

1.3 Relevance/Significance of the project

This project contributes to the following five ways:

1- Defense in-depth pipeline for shell commands generated by LLM.

The five levels of operation work together in such a way that if one fails, the others address it. All the levels of operation work together in that if one fails, the others will address it.

2- Host-aware prompting through lightweight SSH probing.

Each time ShellSentry receives an instruction to generate a new generation, it collects per-host snapshots (`uname -a`, `active systemd units`, and `listening TCP/UDP sockets`) in parallel, and passes each snapshot it collects to the LLM, allowing it to choose the tools and paths that are relevant to the services running on each target.

3- RAGBASHAMLoadedGrounding for Bash automation.

Sentence transformers/all-MiniLM-L6-v2 are used to encode a curated knowledge base of trusted pairs of description and command, and indexed using inner-product (cosine) similarity in FAISS. The top-k examples are added to each LLM prompt, which helps to minimise hallucination and makes it easier to trace and audit the generated commands.

4- An execution architecture that is based on an audit-first approach.

Each request, each command, the list of targets, and each result per host are recorded as an ExecutionLog entry in the database and in a file log. Multi-line scripts are automatically saved to the remote host (`$HOME/ShellSentryScripts/`) for later listing, re-execution, and AI-generated explanations using the dashboard.

5- Repeatable assessment for security, performance, and model size.

The system was tested on eight security test categories, four model configurations for the LLM, two different modes of execution (read-only, open), and a reference set against the industry standard Shell-GPT using the same set of natural-language prompts.

1.4 Report Outline

The rest of this report is structured as follows. The literature review and previous systems in Chapter 2 cover NLSA, prompt injection, and Retrieval-Augmented Generation. The project management approach used is described in Chapter 3, along with the Software Development Life Cycle, risk register, and activity plan selected. The requirements are presented in Chapter 4, which includes elicitation requirements, functional and non-functional requirements, as well as a system-level model (architecture and ERD).

The system design is described in Chapter 5, along with the details of the database schema, user-interface design, and pseudocode for the core procedures. The implementation and evaluation results are discussed in Chapter 6, in which security testing, performance measurements, model-comparison experiments, and a comparison of the solution with a real-world solution are presented. Lastly, Chapter 7 provides a summary of the results and recommendations for future research in the project.

Chapter 2

Literature Review

This chapter situates ShellSentry in the current research on Large Language Models (LLMs), Bash command generation, and IT operations automation. The background of systems that translate NL-to-Bash (NL2SH) and the limitations are summarized in Section 2.1. Three recent academic studies are reviewed that are directly relevant to the ShellSentry design, in Section 2.2. The findings are summarized in Section 2.3 and are used to inform our project in Section 2.4.

2.1 Background

Natural Language to Bash (NL2SH) systems enable a user to describe a task in natural language in the context of Linux and then get back runnable Bash commands. The basic administration of Linux involves learning the syntax of the commands and dozens of utilities, making it hard for novices and tedious for advanced administrators when they must repeat the same task on hundreds of hosts. In recent years, with the advent of Large Language Models (LLMs), NL2SH has seen significant enhancements, with the ability to interpret user intent and produce syntactically valid shell commands for a variety of tasks using instruction-tuned models (Yao, 2024).

Despite all these developments, the output of bash generated by AI is still unreliable in three ways (Zhang, 2024). Firstly, models still generate syntax errors and pick the inappropriate tool to do the job. Second, they hallucinate destructive operations: A seemingly innocuous request may result in a command that wipes out data or alters system configuration in ways that aren't intended. Third, although the output may be technically correct, it is not necessarily also functionally correct; that is, two different commands may yield the same output, and a command that looks correct may have a different behavior than the user expected.

Research has therefore recently been refocused on the validation, refinement, and controlled execution of generation. These are the three dimensions addressed by the studies reviewed in Section 2.2, and ShellSentry enhances all three aspects by leveraging LLM-generated Bash, multi-layer validation, secure execution via SSH, audit log, and managed scheduling.

2.2 Related Work

This section summarizes three recent studies that were relevant to the design of ShellSentry: ScriptSmith is an LLM framework for generating and refining Bash scripts for IT operations; An empirical study on LLM-supported Natural Language to Bash translation, and an LLM-as-a-Judge framework for automatic code validation and refinement, respectively. They address the three pillars of secure NL2SH: generation, evaluation, and refinement, as well as the voids they create, which inspire our design decisions.

2.2.1 ScriptSmith — Automated Bash Script Generation, Assessment, and Refinement

ScriptSmith is an automated framework for creating and refining Bash scripts with an LLM designed to target IT operations. It creates the Bash scripts for the incident-remediation task and checks their correctness; if the scripts are not correct, it automatically corrects them; and if they are correct, it saves them in a re-usable catalogue to be reused later. The authors say they achieved measurable gains in the accuracy of Bash generation as well as in manual scripting time in Site Reliability Engineering (SRE) contexts (Chen, 2024).

The major strength of ScriptSmith is that its generation, automated assessment and refinement are an integrated process, rather than being elements of a separate process, and that the catalogue can be reused to cut down on the duplication of effort for recurring incidents. The framework shows some promising results for using LLM outputs for shell automation in conjunction with an evaluation step.

ScriptSmith also has definite limitations. The framework focuses on generation and evaluation and offers no safe execution; no authentication layer, no SSH-based multi-host orchestration, no audit log of what was executed where, nor is there a scheduling mechanism. The system is also reliant on human expert input prior to the introduction of scripts in production, reducing its degree of operational autonomy.

ShellSentry extends ScriptSmith's principles of generation + validation with a defense-in-depth pipeline, authenticated multi-host SSH execution, an Execution Log audit trail, and a Safe Cron Mode for managed scheduling.

2.2.2 Leveraging machine learning for natural language to bash translation (LLM-Supported NATURE)

In this study, the task is by itself: Given the translation output of an LLM, how do you determine if it performs the same action as the user's command would? The authors build a verified set of instruction-command pairs and introduce a functional-equivalence heuristic which uses LLM assessment of outputs generated from executing commands to determine if they are equivalent, yielding a high accuracy rate in deciding whether two commands execute the same task in Bash.

The element that this paper does best is its evaluation methodology. The work espouses and proves empirically that it is not possible to measure the correctness of Bash only syntactically, but also functionally, since there are several different utilities that can perform the same action. These are also the most frequently occurring failure modes of LLM in this area, as documented in the paper: incorrect formatting, wrong utility selection, and invalid Bash syntax (Yao, 2024).

An important security observation made in the paper is the direct motivation for the design of ShellSentry: the issue of how to make the process of removing a shell more difficult. The authors also noted that during testing, they encountered commands that were hallucinated by the LLM, which caused harm to Docker environments, making it clear that LLM-generated Bash should not be run until it has been validated. On the more complex models, error rates remained unfavorably high for unattended deployment.

The study's restriction is that it is still a benchmarking and translation study and not a platform for operation. Doesn't cover authentication, remote server management, or secure deployment.

This research confirms the security-first design of ShellSentry, where the commands generated are validated and limited by a whitelist, blacklist, and a read-only policy before they are sent to a remote host.

2.2.3 LLM-as-a-Judge for Automatic Code Validation and Refinement

The framework of this paper is an LLM-as-a-Judge approach to validate and improve bash commands from natural text. Using bidirectional functionality matching and logic representation techniques, the framework examines whether the generated code is logically correct in achieving the desired task, not based on string similarity. A reflection-agent loop then automatically rewrites the wrong commands, and the authors claim to have used it to gain considerable accuracy in the command refinement process (Zhang, 2024).

The strength of this work is in its semantic validation approach. Logical equivalence is a better correctness signal than text similarity, and the reflection mechanism makes the one-shot generation step

into a self-correcting loop. The paper has very good arguments for NL2SH systems to consider validation not as an add-on, but as an integral feature.

The downside of the whole approach is that it relies on the LLM's reasoning. The judge can also hallucinate or misinterpret the task, leading to a misjudgment of the verdict and the refinement itself. The study also does not go into secure deployment, secure authentication, SSH orchestration, and audit logging.

This work validates the approach that ShellSentry takes toward validating each command that is generated before it is executed, and only commands that pass all layers are passed to the SSH executor.

2.3 Summary

The reviewed studies lead to five conclusions:

Although LLMs may be able to reliably produce Bash, they are not sufficiently robust for production use without supervision.

The correct measure of correctness of the NL2SH output is functional correctness, not syntactic similarity.

Validation & refinement loops significantly enhance the quality of generations.

In real test environments, hallucinated and destructive commands have been seen, and it is empirically shown that combinations of deterministically checking safety must also be included in NL2SH.

Existing literature focuses on generation and evaluation, while security aspects of operational deployment (authentication, multi-host orchestration, audit logging, scheduling, and others) remain poorly documented.

2.4 Relevance to Our Project.

The ideas from each of the above studies are carried forward and expanded on in ShellSentry. It is based on the closed-loop generate → validate → refine of ScriptSmith, and adds secure remote execution capabilities, command-policy enforcement, audit logging, and operational management capabilities (the operational layer that ScriptSmith leaves to users) of the scheduling system.

The conclusions of the LLM-Supported Natural Language to Bash Translation study apply this to the evidence that LLM output needs to be checked for correctness before use, as seen in practice with hallucinated destructive commands, justifying ShellSentry's whitelist, blacklist, per-command handlers, and read-only execution policy.

It adopts the principle that validation should be a first-class part of the pipeline from the LLM-as-a-Judge framework and operationalizes this principle as a deterministic and layered validator instead of a single LLM judge, avoiding the danger of an LLM hallucinating a valid but unsafe command. ShellSentry aims to be a secure-execution problem, while previous work considers NL2SH as a generation-and-evaluation problem.

What sets this project apart from the existing literature is that it is an integrated, defense-in-depth platform that combines natural-language input, host-aware prompting, retrieval-augmented grounding, multi-layer command validation, authentic multi-host SSH execution, audit logging, script archiving and re-execution, and all of it is executed securely, transparently, and reproducibly across a wide variety of operational environments.

Chapter 3

Project Management

The process for planning and executing the ShellSentry project is described in this chapter. Chapter 3.1 introduces the software process model and provides an explanation of how the model was used in the development of this prototype. Project risks and project mitigation strategies are identified in Section 3.2. The artifacts of the activity plan and scheduling are presented in Section 3.3 for delivery of the system within the time frame of the senior project.

3.1 Process Model

The Waterfall process model was chosen as the main software development process model. Waterfall is an approach to development that follows sequential stages, with each stage creating approved artifacts before moving on to the next (requirements, design, implementation, testing, and deployment/documentation). This is the right structure for an academic senior project for a variety of reasons.

First, clear checkpoints and stable requirements documentation, particularly for Chapters 4 and 5, are needed by the department and supervisor, and Waterfall supports this through formal phase outputs. Second, ShellSentry meshes multiple delicate issues—remote execution, large language model (LLM) generation, and security policy enforcement—that are best addressed through threat and requirement thinking before detailed design and coding take place. Third, the fixed calendar from Sept. 2025 to May. 2026 aligns naturally with discrete phases with predictable milestones (requirements sign-off, design freeze, feature-complete build, test campaign, final submission).

The model was used pragmatically: some minor changes to the requirement were accepted after implementation, if they did not break the architecture (such as more precise validation messages or UI texts). But core boundaries of functionality (authenticated execution, remote running via SSH, validation layers, audit logging, script archive, and safe cron workflow) were kept as baseline scope, consistent with a Waterfall approach, before intensive coding began.

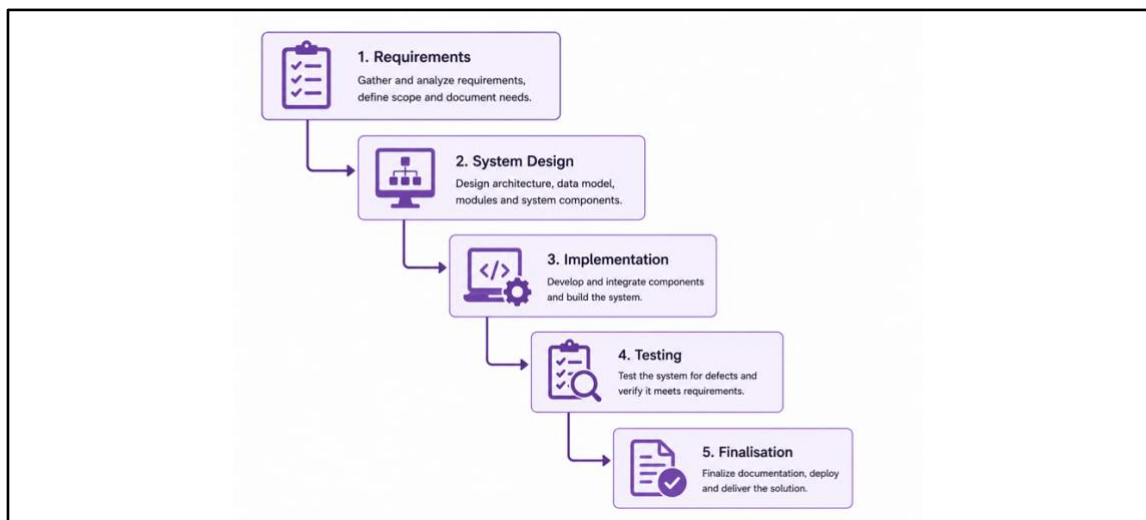


Figure 1: Phases of the waterfall model used for ShellSentry

3.2 Risk Management

The project talks about risk management practices to identify, evaluate, and reduce potential risks in Section 3.2 that can impact the performance, security, and reliability of the system. To ensure stable workflow execution, protect sensitive information, and prevent operational failures, it is necessary to use risk management. Technical and security-related risks include API instability, SSH connectivity issues, destructive command execution, prompt injection attacks, cron schedule abuse, and credential exposure. Relevant mitigation measures are suggested to reduce these risks and keep the system safe.

Table 1 provides a risk management analysis of the project and details the identified risks, their levels of severity, potential impacts, and mitigation strategies. The table illustrates the impact of preventive measures like command validation, secure credential handling, audit logging, retry mechanisms, connectivity checks, and strict execution controls on vulnerabilities and the overall stability and security of the system.

Table 1: Identified risks and mitigation plans for ShellSentry

Risk	Level	Impact	Mitigation
LLM API instability (timeouts, rate limits, provider outages)	Medium	Generation failures block the main workflow	Client-side retries/backoff; graceful error responses in the UI; fall back to regex-based routing where applicable; keep logs for diagnosis
SSH connectivity and credential issues (keys, passwords, firewalls)	Medium	Partial or failed multi-host runs	Pre-flight reachability checks; parallel execution so one host does not block others; documented .env setup
LLM generates state-changing or destructive commands (e.g. <code>rm -rf</code> , <code>mkfs</code> , <code>dd if=...</code>)	High	Data loss or host compromise	Command validator (whitelist/blacklist/restricted patterns); read-only mode default; normalization before execution; audit logging
Prompt injection — malicious or misleading natural-language input crafted to bypass intent or policy	High	Unsafe execution or policy bypass	SecurityLayer input checks; intent routing + regex blocks (e.g. destructive cron phrases); validation after LLM output; no blind trust of model text
Cron schedule abuse — scheduling archived scripts too aggressively, wrong expressions, or	Medium–High	Unwanted automation or denial patterns	Safe Cron mode; list/schedule only for managed entries; destructive removal phrases blocked; strict cron expression

attempting bulk removal of crontab			checks; script must exist and pass re-validation
SSH credential theft or exposure (passwords/keys in logs, screenshots, or misconfigured .env)	High	Unauthorized access to lab servers	Secrets only in environment/config (not in repo); avoid logging secrets; document secure handling; least-privilege SSH user for demos

3.3 Project Activities Plan

The activity plan spans 34 weeks, from Monday 15 September 2025 to Monday 11 May 2026, to match the official teaching time period and submission deadline for the senior project.

Each of these workstreams was carried out in the ‘Waterfall’ style, meaning that, at each stage, something clear was produced before moving on to the next: (1) requirements and planning: scope, security boundaries and lab constraints clarified; (2) system design: architecture, data model, and execution flow; (3) implementation: building and integration of Flask application, LLM and SSH components, script archive and safe cron features; (4) testing and evaluation: functional runs on various hosts, security checks, performance observations, report; (5) finalisation: completion of Chapters 5-7, figures and references, poster, submission materials, including any department requirements for appendices, deliverables, etc.).

The following table and Gantt chart show how these workstreams were planned throughout the project timeline.

Table 2: ShellSentry project activity plan

Phase	Period (inclusive)	Main outputs
P1 – Requirements & planning	15 Sep 2025 – 26 Oct 2025	Problem refinement; initial objectives and scope; environment setup; draft requirement list; preliminary architecture sketch
P2 – System design	27 Oct 2025 – 21 Dec 2025	Architecture diagram; module responsibilities; database design (User, ExecutionLog); UI wireframes; pseudocode for execution pipeline
P3 – Implementation	22 Dec 2025 – 29 Mar 2026	Working Flask application; authentication; /api/execute orchestration; SSH executor; LLM client + RAG integration; archive and safe cron paths
P4 – Testing & evaluation	30 Mar 2026 – 26 Apr 2026	Functional tests on multi-host scenarios; security-focused tests; performance timings on core procedures; results tables for Chapter 6
P5 – Finalisation & documentation	27 Apr 2026 – 11 May 2026	Final report chapters and revisions; figures and references polish; poster/demo preparation; submission checklist (including appendices/CD requirements per department guidance)

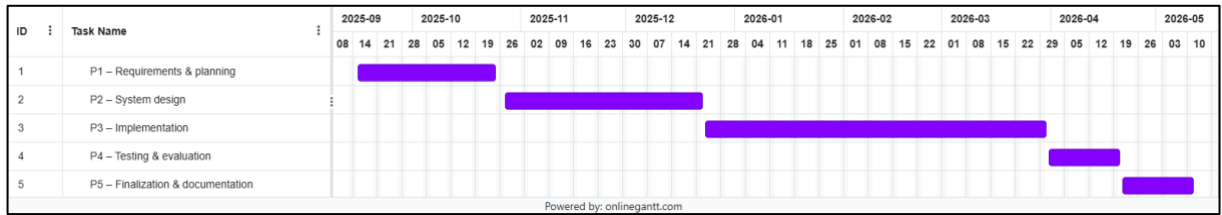


Figure 2: ShellSentry project Gantt chart

3.4 Chapter Summary

In this chapter, the project management process followed in the development of ShellSentry was discussed, including planning, risk management, and scheduling activities, from the concept to prototype. The Waterfall software development model was chosen as the main process framework because of its structured and defined nature, and its ability to satisfy the needs of academic projects, which have clear deliverables and milestones to meet. The chapter also discussed the key technical and security challenges encountered with the project, such as LLM instability, unsafe AI-generated commands, SSH connectivity failures, prompt injection, cron misuse, and credential exposure, as well as mitigation measures for these risks to minimize their impact. Finally, the project activity plan presented the project development timeline for the requirements analysis, system design, system implementation, testing, and documentation for the prototype of ShellSentry within the time frame of the senior project.

Chapter 4

Requirement Collection and Analysis

This chapter outlines the requirements of ShellSentry and how the requirements were identified. It starts with requirement elicitation (Section 4.1), which is a summary of how the stakeholders' input and technical constraints helped to develop the specification.

The results of that process are expressed as functional and non-functional requirements in Section 4.2. Section 4.3 introduces models of the systems—the high-level architecture of the web application, the validation layers, the LLM and SSH services, and the relational schema that will support authentication and audit logging.

These artefacts collectively serve as a stepping stone leading from the problem statement in Chapter 1 to the design and implementation chapters that follow.

4.1 Requirement Elicitation

Two participants were interviewed using semi-structured interviews to collect security, traceability, and safe automation requirements. Participant 1 responded to the question(s) in writing from a Cyber Security viewpoint. In the interview, Participant 2 provided a response on each of the themes with short numbered statements instead of lengthy prose.

The interviews provided ShellSentry with a focus on defence in depth, read-only defaults, deterministic validation, and auditability. The questions and responses from the interviews have been summarised in Table 4.1.

Table 3: Requirement elicitation — questions and participant responses

NO.	Question	Participant 1 (Umer Khan)	Participant 2 (anonymous)
1	What are the main risks when manually executing Bash commands across multiple remote servers?	Risks include executing commands against the wrong host or environment, privilege misuse, lack of validation/safeguards, inconsistent execution across the fleet, and insufficient logging and auditing. Even skilled administrators can cause outages or security exposure with a single mistyped command.	The biggest risks are running high-privilege commands without proper verification, and lack of clear traceability for what changed on each system.
2	How does human error contribute to incidents in system administration?	Human error remains a major cause of operational and security incidents—e.g. accidental deletion or modification of critical files, incorrect permission changes, running commands with elevated privileges, exposing secrets in scripts or logs, and misconfigurations that weaken	Human error is a leading cause of incidents: harmful commands, permission misconfigurations, and running commands in the wrong environment (e.g. production instead of testing).

		security. Often the issue is scale, complexity, and time pressure, not only lack of skill.	
3	What new risks appear when an LLM is used to generate shell commands?	New risks include prompt injection, unsafe command generation, hallucinated syntax or logic, hidden chaining of dangerous operations, privilege escalation opportunities, and indirect trust in untrusted input. The output is probabilistic, which changes the security model compared with traditional automation.	Introducing an LLM adds risks such as prompt injection, and commands that look valid but can still cause serious system impact.
4	In practice, which threats are most realistic for LLM-assisted shell execution?	Privilege abuse and unsafe command execution are among the most immediate risks—a technically valid command can still be harmful if run with excessive permissions. Prompt injection is highly relevant when the model processes external or untrusted content; command injection matters if user input is concatenated into shell commands without strict sanitisation and parsing.	Privilege abuse and command injection are among the most realistic threats—a small mistake with elevated privileges can quickly become a serious incident.
5	Is whitelist/blacklist validation enough on its own to secure generated commands?	Not sufficient. Blacklists are especially weak because bypasses continually appear. A safer design combines strict allowlists, parameterised templates, RBAC/least privilege, sandboxing, human approval, simulation/dry-run, strong auditing, and defence in depth.	Whitelist/blacklist checks are useful but not sufficient alone; they should be combined with sandboxing and least-privilege access.
6	How important is read-only-by-default behaviour for AI-assisted administration?	Extremely important. Read-only default dramatically reduces blast radius and is one of the best architectural decisions for such systems. Destructive or state-changing operations should require elevated approval, extra validation, and explicit operator confirmation.	Highly important—it significantly reduces risk and keeps the system focused on visibility and analysis rather than unchecked modification.
7	What logging and auditing should stakeholders expect before trusting this	Expect immutable audit logs, user attribution, timestamps, original prompts, generated commands, approval history,	For a SOC-style context, expect logging of operator identity, IP addresses, timestamps, and execution outcomes

	class of system (e.g. in an enterprise or SOC-oriented context)?	execution results, target-host tracking, rollback/recovery visibility, and alerting for high-risk operations. Traceability and accountability are critical.	(success/failure). <i>(Participant 1 gave a broader enterprise list above.)</i>
8	What minimum security controls would you recommend for a responsible educational proof of concept?	Use sandboxed execution only, no production access, least-privilege execution, read-only default, strict command allowlists, human approval before execution, complete logging, and clear documentation of limitations. Demonstrate blocked malicious prompts in evaluation to show security thinking, not only happy-path behaviour.	Minimum controls should include: execution in an isolated sandbox or container, strictly limited privileges, blocking dangerous commands by default, full execution logging, and an approval step for sensitive operations.

Convenience is not an excuse for controls: validation layers, least privilege, defaults – read only, and traceable execution – are central. These themes align directly with the functional and non-functional requirements from ShellSentry, as presented in Section 4.2 (e.g., authenticated execution, deterministic policy enforcement after LLM generation, read-only configuration, persistent execution logs).

4.2 System Requirements

This section specifies the functional requirements and the non-functional requirements. The description of functional items is a description of what the system does, and the description of non-functional items is a description of how well it must do it within a constraint that is relevant to security-sensitive automation.

4.2.1 Functional Requirements

- Natural-language to Bash generation

The system shall use a Large Language Model (LLM) to transform a user's natural-language request into a Bash command or multi-line Bash script that can be executed. Only executable shell commands should appear in the output, and it should be suitable for being executed on remote Linux hosts.

- Host-aware context generation

The system should be able to probe through SSH for selected Linux hosts and gather data like operating-system information, running services, listening network sockets, etc. This contextual information needs to be provided to the LLM along with the user request to enhance the generation of commands that take the environment into account.

- Retrieval-augmented grounding

For each query, the system will use semantic similarity search to find the top-k semantically similar examples of commands in a curated Bash knowledge base and add them to the prompt for the LLM to reduce hallucinations and ensure consistency with trusted command patterns (Sentence Transformers Documentation, 2026).

- A deterministic safety validation prior to execution

All commands output by the system shall be checked for deterministic policy enforcement, such as whitelists, blacklists, restricted-pattern detection, and optional read-only execution policies, before being executed as well as any script stored in the archive. Any command that does not validate shall not be executed.

- SSH execution

The system shall begin to run approved Bash commands and scripts remotely on selected Linux hosts via SSH connections (Ylonen, 2006). The execution subsystem shall provide the capability for pre-flight reachability checks and isolated per-host execution results, to ensure failure on one host will not prevent failure on another host.

- SSH authentication management

The system shall provide support for multiple SSH authentication mechanisms such as password-based authentication, SSH private-key authentication, SSH-agent usage, and credential overrides as configured on the system per server.

- Multi-server orchestration

The system shall allow concurrent execution across multiple configurable Linux hosts, defined by environment-based configuration. To minimize delays due to sequential execution, the system shall execute commands in parallel.

- Script archive and re-executes.

The system shall automatically save generated multi-line Bash scripts on the remote hosts to be reused later. The system shall allow for archived scripts to be listed, validated prior to execution, re-run the saved scripts, and retrieve script content for explanation.

- Safe Cron scheduling

The system shall support cron scheduling for archived scripts with control by providing a list of managed cron entries and a mechanism to schedule existing archived scripts by specifying a cron expression that is validated. Any destructive operation on cron, such as the removal or emptying of crontab entries, shall be prevented (The Linux Documentation Project, 2026).

- Ensure that it is properly audited and reported. Properly audit and report it.

The system shall be able to store the original natural-language request, the command generated, the target servers selected, the execution status, times, and results of executed commands persistently. The system shall also produce technically structured reports and summaries in a human-readable form of executed operations.

- Executive and Security policies are settable.

Execution policies shall be supported in the system via configuration by the environment, such as read-only execution enabled by default with configurable override support, privileged/root execution disabled by default with configurable administrative override, configurable archive behavior, configurable cron management settings, and configurable LLM provider settings.

4.2.2 Non-functional requirements

- Security

Defense in depth: input sanitization (Security Layer), LLM-assisted and regex-based intent routing with hard blocks for destructive cron phrasing, policy validation after generation, read-only execution enabled by default with configurable override support, and SSH execution under configured credentials. Secrets will be stored in the environment configuration and not in sources.

- Confidentiality and least privilege

No SSH credentials, API keys shall be stored within the repository, and remote execution shall be performed with the smallest OS privileges granted in the deployment documentation.

- Reliability and resilience

The LLM client should have any API failure that is transient (e.g., timeouts, rate limits) retried/backed off. SSH execution will allow some hosts to fail and will display in API responses with a clear wording of any per-host failures.

- Performance and scalability (within the limits of the prototype)

Host probing and execution should be done concurrently to prevent sequential bottlenecks. Latency is limited by LLM and network behavior; the architecture should not add unnecessary serial stages after policy checks.

- Usability

The web dashboard shall display a natural-language description, extended by AI explanations (optional), and a technical report that can be expanded to provide technical detail, for easy interpretation for both technical and non-technical users.

- Maintainability and auditability

The codebase will be structured to isolate routing, validation, SSH, LLM, RAG, and formatting issues, with all execution paths being logged and documented in a database for post-incident analysis.

4.3 System Models

This section introduces key algorithms that ShellSentry employs. These algorithms work well together to ensure secure natural-language processing, command validation, remote execution, script management, and retrieval-augmented generation. They offer a combination of controlled AI-driven remote administration, deterministic security validation, and operational traceability.

4.3.1 System architecture

Major components and control flow of ShellSentry are shown in Figure 4.1. End-to-end flow: The authenticated user types of natural language and an optional list of the servers he/she wants to access, and posts that to the browser, which then passes them on to Flask (POST /api/execute). The Security Layer checks raw text. When configured, intent classification (resolve_execute_route) is done with an LLM classifier, otherwise using regex, and includes a safety block that prevents forbidden cron behavior. This depends on the task type: either SSHExecutor is called for managed cron / script-archive operations, or the standard pipeline is used: probe_host_context -> RAG retrieval -> LLM generate_command -> CommandValidator -> normalize_for_execution -> execute_on_servers. The result goes through the result formatter; result summarizers and script explainers are optional and helpful. SQLite is used to store data for users and the ExecutionLog.

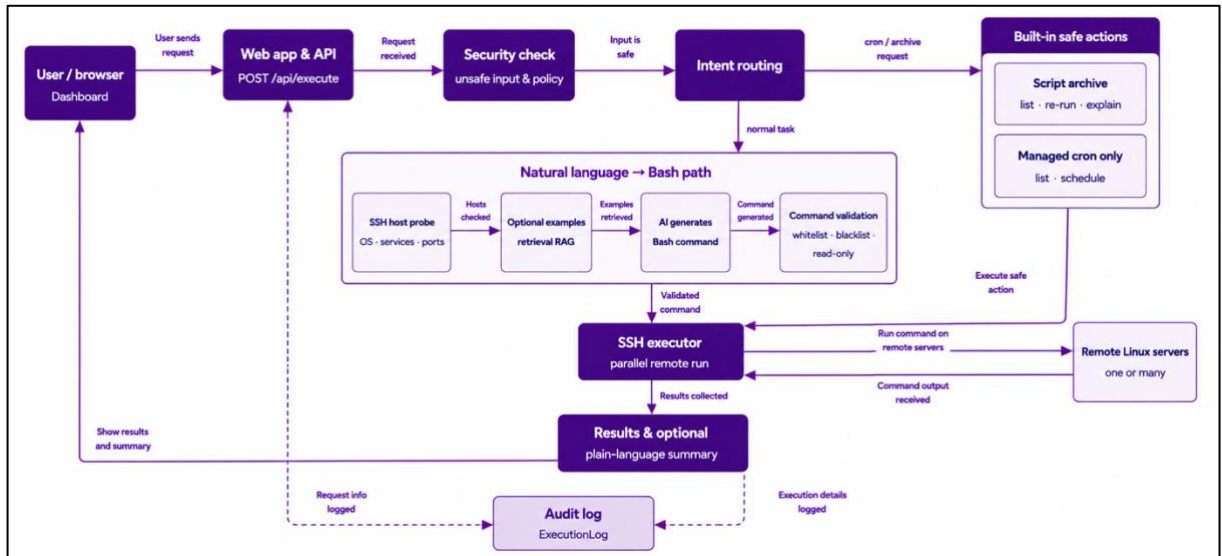


Figure 3: System architecture

4.3.2 Entity relationship diagram (ERD)

ShellSentry is based on a relational database for authentication and audit. The logical schema is given in Figure 4.2. User stores username, email, hash password, is_admin, created_at.

For each run, ExecutionLog contains the original_request and the generated_command, as well as the denormalised username, the target_servers (which are serialised), the results of the execution (execution_results), the status of the execution (execution_status), and the time of the run (timestamp), with user_id as a foreign key on User.

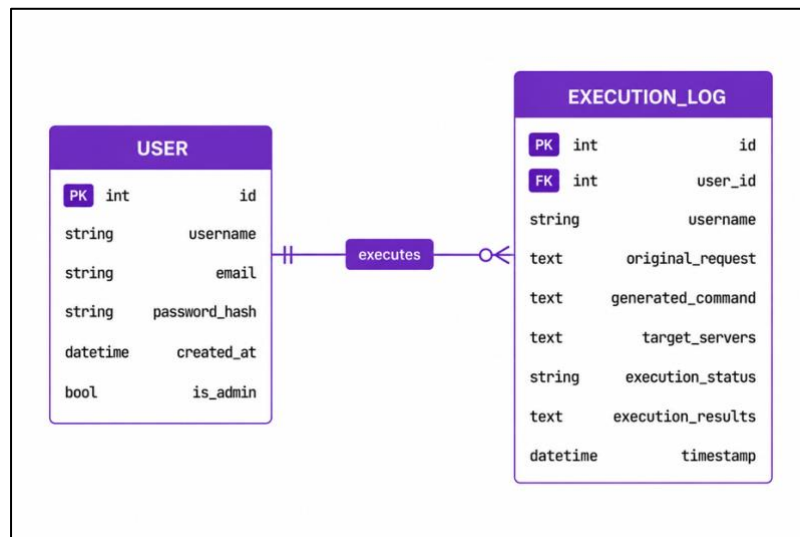


Figure 4: Entity relationship diagram (ERD)

4.4 Chapter Summary

The requirement analysis for ShellSentry was presented in this chapter, which covered the elicitation of requirements from the stakeholders, functional and non-functional requirements, and the key models of the system. System requirements were directly influenced by the elicitation process with regard to security, validation, auditability, and controlled automation. The chapter also presented the high-level system design and database design, which will form the basis for the detailed design in Chapter 5 and the design implementation and evaluation in Chapter 6.

Chapter 5

System Design

5.1 Introduction

In this chapter, we discuss the logical design of ShellSentry before we discuss implementation and evaluation in Chapter 6. Designing the system to address both functional and non-functional requirements that were identified in Chapter 4, this chapter presents three complementary views: first, the set of persistent data managed by the web application (Section 5.2); second, the set of user-facing interfaces and navigation structures (Section 5.3); and third, the set of core procedures and algorithms to support natural-language routing, validation, remote execution, and reporting (Section 5.4).

The ShellSentry architecture is designed to be modular. The Flask application splits the following into different parts: HTTP orchestration (routes), intent resolution (`execution_router`), input hygiene (`SecurityLayer`), LLM interaction (`LLMClient`), retrieval grounding (`RagPipeline`), deterministic command policy (`CommandValidator`), parallel SSH operations (`SSHExecutor`), response shaping (`result_formatter`), and persistence (models). For command effects, as well as for saved scripts in the configurable directory (such as `$HOME/ShellSentryScripts`), remote Linux hosts continue to be the system of record; for authentication and auditability, the local database is the primary system of record, as found in the interview.

The prototype's relational schema is described in Section 5.2. The remaining part of this chapter (sections 5.3 and 5.4) explains the interface design and algorithmic behavior that implement the concepts of defense in depth, read-only defaults, and auditable multi-host execution, as explained in Chapters 1 and 4.

5.2 Database Design

This section illustrates the tracing of system requirements from Chapter 4 to the design elements given in Chapter 5. The section documents how the functional and non-functional requirements were mapped to specific architectural modules, interfaces, and algorithms, with the intent to structure the proposed design to meet the project objectives and still remain secure, maintainable, usable, and AI-assisted remote-execution controlled.

5.2.1 Purpose of the database in ShellSentry

The prototype runs on a relational database ([Flask-SQLAlchemy](#) with configuration provided by the `SQLALCHEMY_DATABASE_URI`, which defaults to `SQLite` when running in development) ([Flask-SQLAlchemy Documentation, 2026](#)) ([SQLite Documentation, 2026](#)). The database is used in two ways that correspond to two of the non-functional requirements of Chapter 4: security and auditability.

It first holds account details for the web tier, like user registration data and password hashes that are used to enable Flask-Login session authentication for users, only to invoke execution APIs. Second, it maintains an execution audit trail of each remote operation that is logged via the application, which includes the user who initiated an action, the natural language prompt provided, the command string accepted for execution (or built-in operation recorded), the set of hosts executed, aggregate status, and a structured per-host result where appropriate.

This database is not designed to be updated with operational state on remote servers, e.g., live process list, disk contents, or the complete collection of archived scripts. That boundary ensures that the local schema is small enough and sensitive duplication is reduced, and matches the Local Educational Prototype Scope stated in Chapter 1. Script storage and crontab (cron) definitions are still stored on target hosts, accessible via SSH paths, and under the user's crontab, as provided by the SSH executor

layer, and are still working as expected. As provided by the SSH executor layer, long-term script storage and crontab (cron) definitions are still stored on target hosts under SSH-accessible paths and under the user's crontab and are still working as expected.

5.2.2 Logical schema and data dictionary

The logical schema consists of two entities: User and ExecutionLog, with a one-to-many relationship between them, meaning that each user can have many rows of execution log. They are related as shown in Figure 4 of Chapter 4.3.2. Field-level metadata is summarized in Tables 4 and 5.

Table 4: User entity — data dictionary

Attribute	Type (logical)	Constraints	Description
id	Integer	Primary key	Surrogate identifier for the account.
username	String(80)	Unique, not null	Login name displayed in logs and UI.
email	String(120)	Unique, not null	Contact identifier used at registration.
password_hash	String(255)	Not null	Stores a werkzeug password hash; plaintext passwords are never persisted.
created_at	DateTime	Default UTC now	Approximate account creation time for administration.
is_admin	Boolean	Default false	Reserved flag for administrative distinction if extended beyond the prototype.

Table 5: ExecutionLog entity — data dictionary

Attribute	Type (logical)	Constraints	Description
id	Integer	Primary key	Unique identifier for each logged execution event.
user_id	Integer	Foreign key → User.id, not null	Links the row to the authenticated principal.
username	String(80)	Not null	Denormalised username for readable audit exports without joining.
original_request	Text	Not null	User's natural-language request as submitted to the API.
generated_command	Text	Not null	Command string or descriptive label for built-in routes (e.g. listing cron or scripts) recorded as executed for audit.
target_servers	Text	Not null	Serialised representation of the host list (JSON string in implementation).
execution_status	String(20)	Not null	High-level outcome category (e.g. success, failed, partial) for filtering and reporting.

execution_results	Text	Nullable	Serialised per-host outcomes and detail (JSON string in implementation).
timestamp	DateTime	Default UTC now	Wall-clock time of logging for chronological audit review.

The relationship between User and ExecutionLog is required on the child side: each row in ExecutionLog corresponds to a single row in User. This would typically involve a policy decision (cascade delete versus keeping anonymised logs), but the prototype makes the assumption of standard relational integrity as enforced by the ORM and the Database.

5.2.3 Design decisions and scope boundaries

For the prototype in the senior project, SQLite (or any URL given via configuration) should suffice, as it is not intended to be large enough to require a multi-process deployment, and the number of rows in the ExecutionLog isn't large enough to set a limit. Typical production deployments would generally move to a server-managed RDBMS and then restrict access to and backups of audit tables.

Some of the data in these fields is stored as text-serialised structured data, as this offers better performance for implementation, and better exports for the report and appendix text data. Lastly, there are no LLM API keys, SSH passwords, or private keys stored in the database – as assumed in Chapter 1 and in the security requirements in Section 4.2.

5.3 User interface and interaction design

The ShellSentry front end is made up of vanilla JavaScript on the dashboard for asynchronously making POST /api/execute calls and server-rendered HTML (Jinja2 templates). The principle of interaction design is that execution is not anonymous: All operational screens must have an authenticated session. The following figures should be filled in with screenshots or wireframes of your working prototype.

5.3.1 Landing, login, and registration

The landing route (/) and the index.html route are for unauthenticated users and redirect authenticated users to the dashboard route. Login (/login, login.html) captures username and password, and on unsuccessful login, it displays a flash message but does not show which field was wrong. Registration (/register, register.html) takes in the username, email, password, and confirmation, and the server checks for the uniqueness of the username, password, and confirmation before adding a record to the User table with a password that has been hashed. When registration is successful, it redirects to login with a success message. These views will meet the prerequisite of authenticated access for all execution paths and the requirement of audit logging by associating future ExecutionLog rows to a known principal.

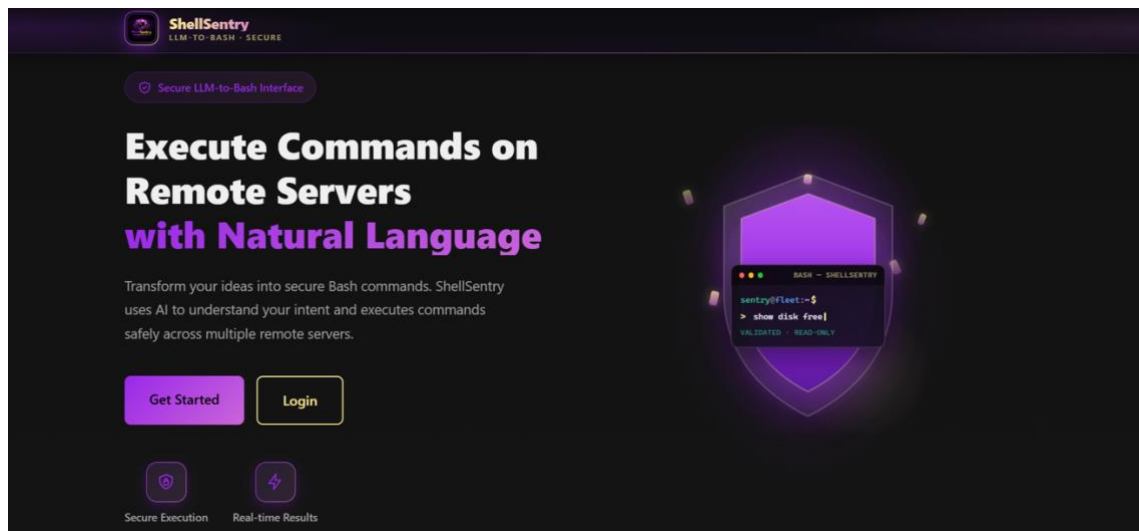


Figure 5: Main Page User Interface

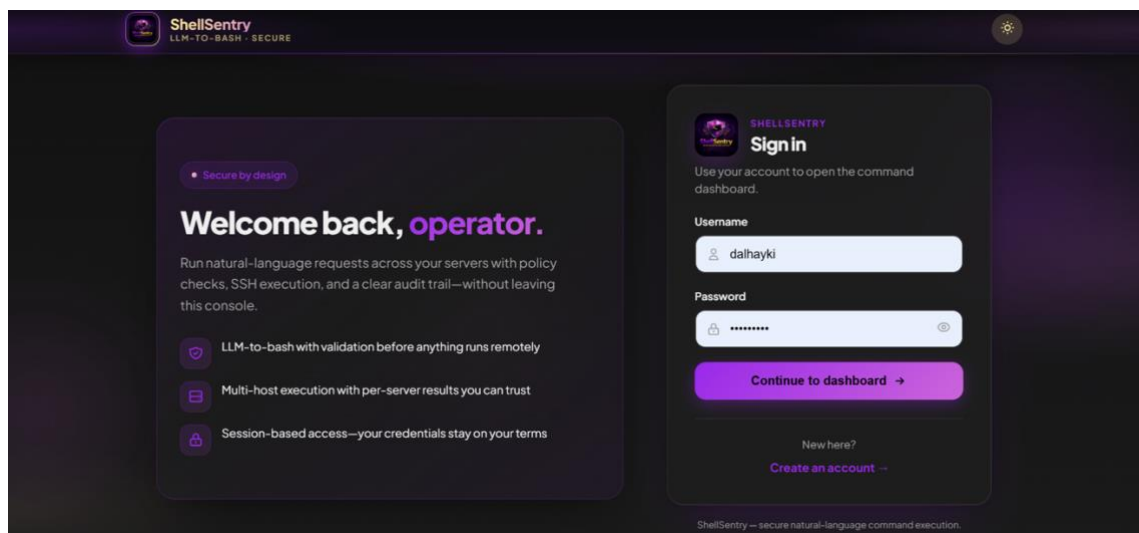


Figure 6: Log-in Page User Interface

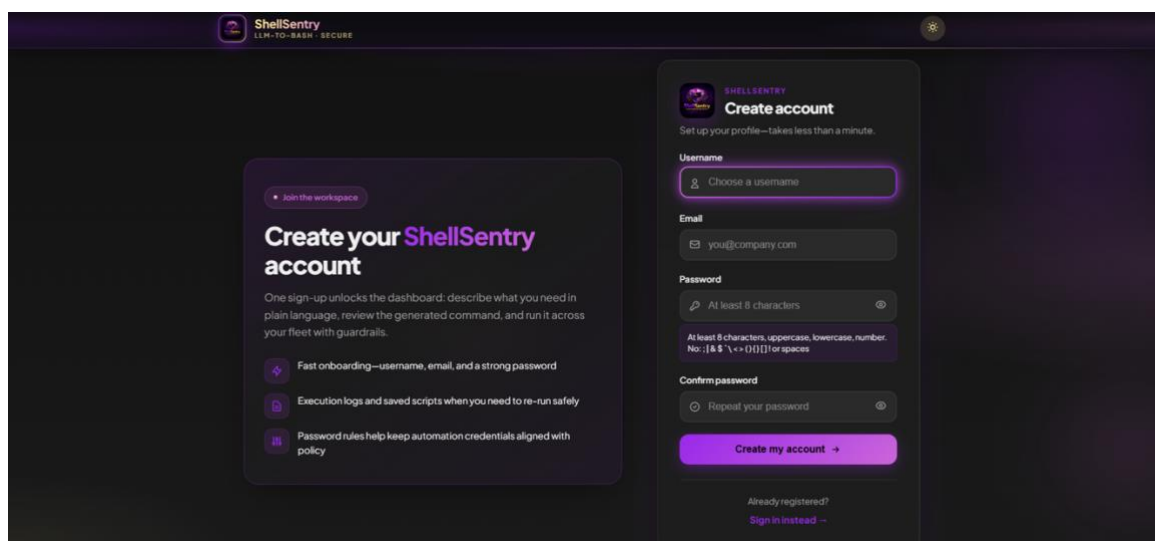


Figure 7: Sign-up User Interface

5.3.2 Main dashboard: request composition and targets

The default workspace is the dashboard (`/dashboard`, `dashboard.html`, `@login_required`). It displays: (1) a text area for the user task, in natural language; (2) an optional target servers field containing a comma-separated list of hostnames or addresses; (3) an execute control, that sends the form to `/api/execute`, with a JSON body `{command, servers}`. If a server list is empty, the client returns an empty list, and the backend fills it in with default servers from environment configuration (`REMOTE_SERVERS`), meeting the multi-server orchestration requirements when defaults are specified. It is possible for the dashboard to contain example prompts (categorized filters in the client script) to assist with discoverability and training use cases.

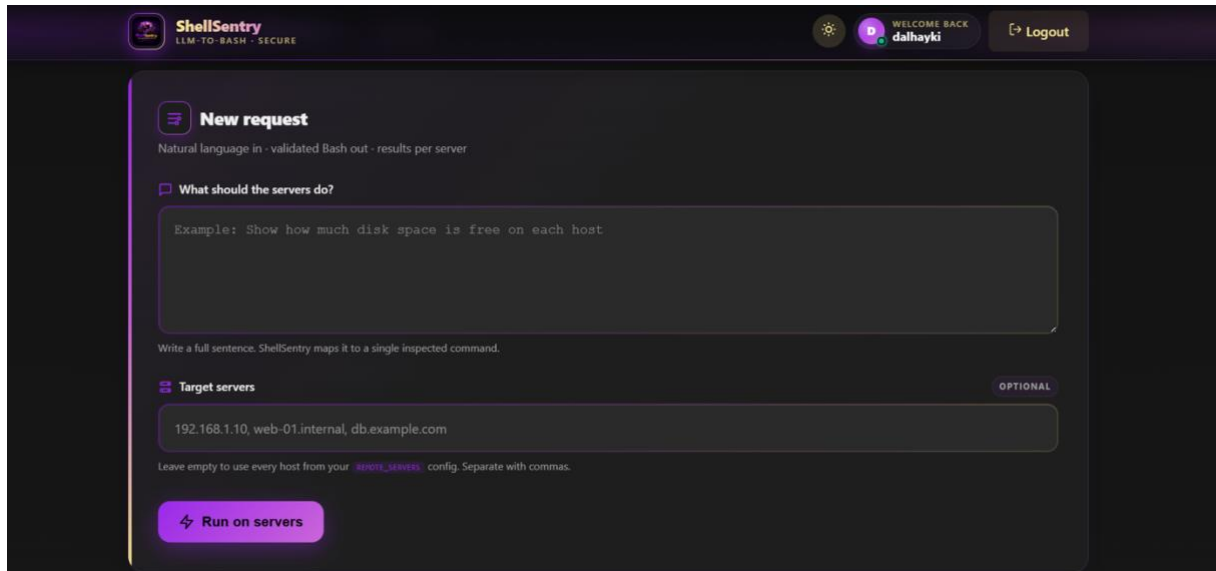


Figure 8: Dashboard User Interface

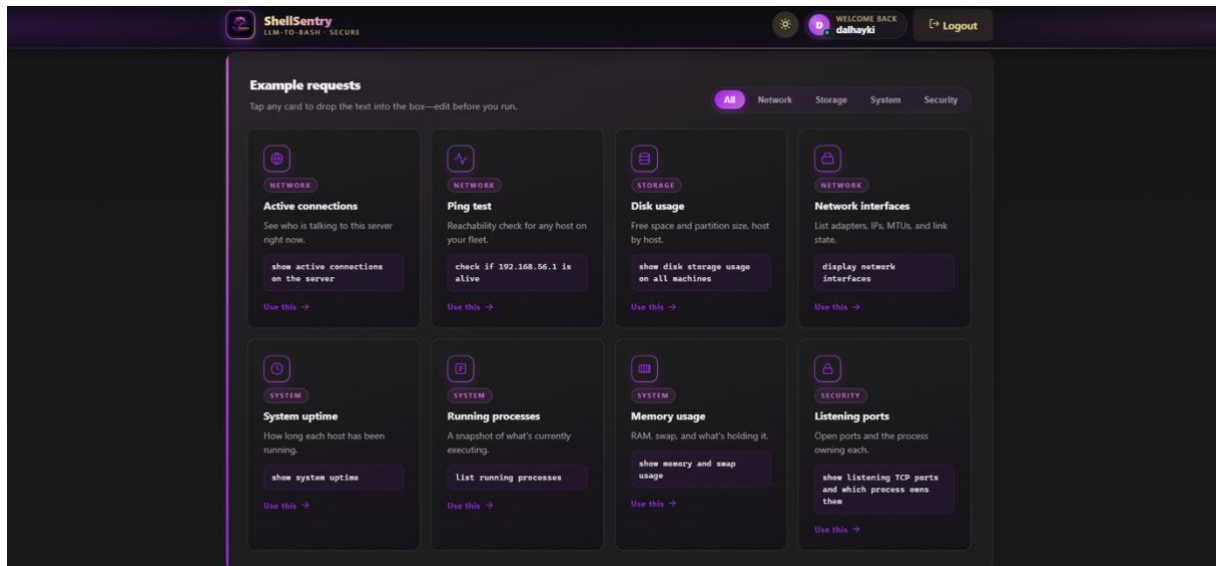


Figure 9: Option Clickable Execution Command User Interface

5.3.3 Results presentation

In cases of success, the client displays a results stack, with the original request, the generated or logical command label, and per-host success/failure counts based on the results object. A plain-language summary (`natural_language_summary`) caters to the needs of non-experts who are interested in reading. If the explanation is present, an optional AI explanation of the technical report (`ai_report_explanation`) appears inside a collapsible disclosure, otherwise a muted fallback message (`ai_report_explanation_error`) is displayed that explains that the technical block should be consulted.

Script explanation (`script_explanation`) is displayed in the same pattern as script archive flows. A copy-to-clipboard control is provided for the technical report (`formatted_report`) to allow for lab reporting and appendices, and is displayed in a collapsible terminal-style panel. This layout is directly used to support the functional requirement of Audit Logging and Reporting at the presentation layer.

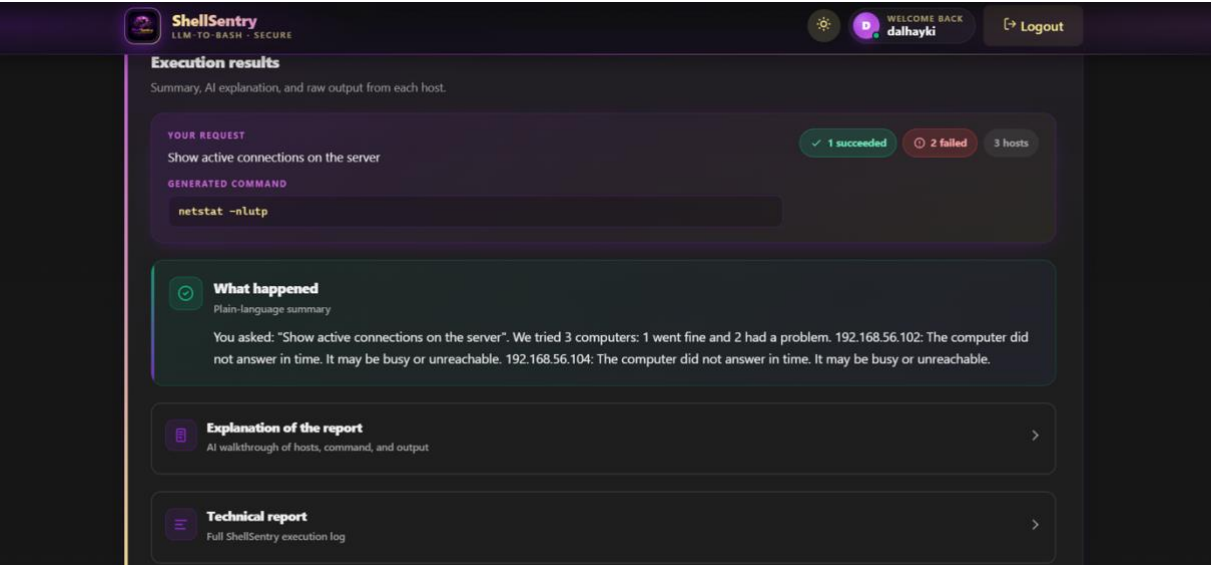


Figure 10: Result User Interface

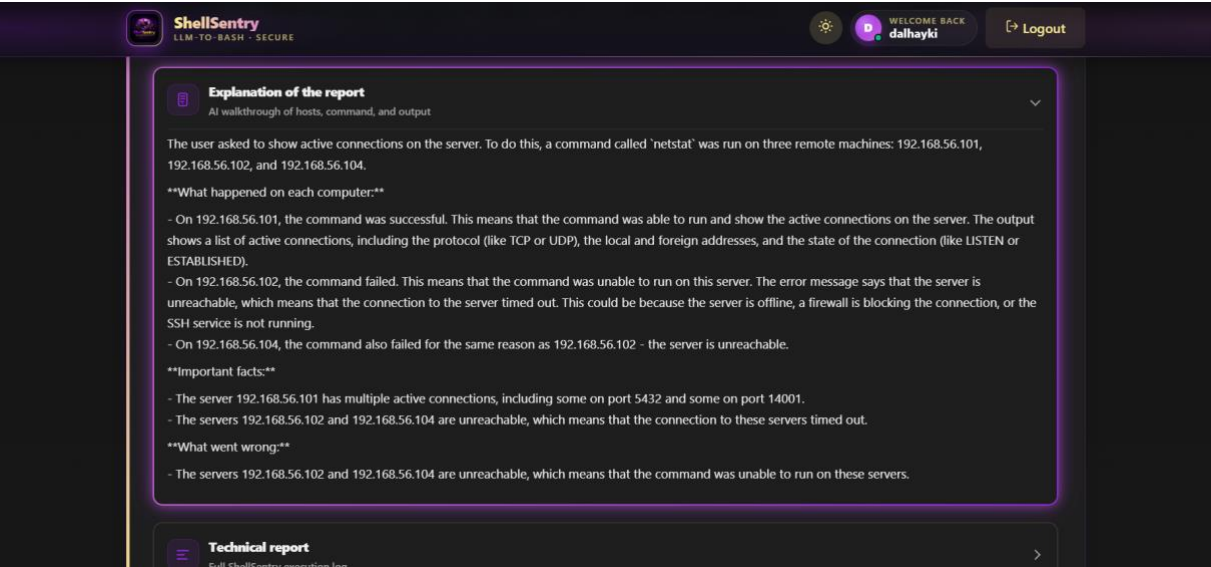


Figure 11: Explanation User Interface

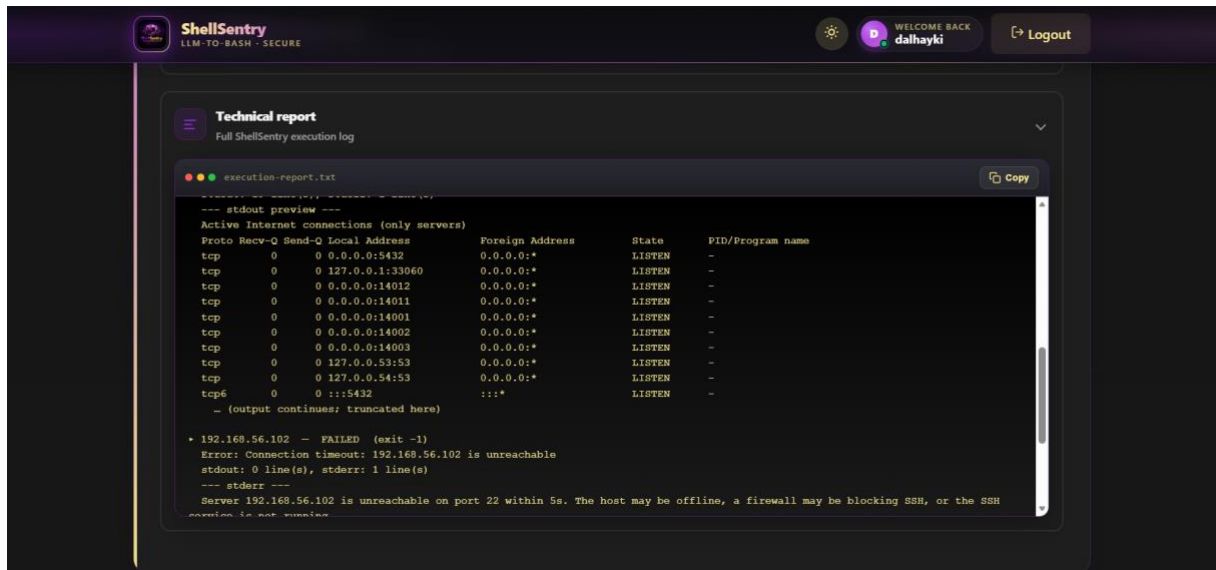


Figure 12: Technical report User Interface

5.3.4 Error and Blocked-Command Feedback

If the client gets an HTTP response with a status other than success, the interface will show a structured error panel, with both human-readable and technical feedback. The system uses the `natural_language_summary` field to explain clearly what is happening and optionally presents validation reasons and backend error details.

If the generated command is not accepted by the validation layer, the interface will show the generated command in a non-editable read-only preview in the blocked/not executed status. The feedback system is able to handle a variety of failure cases, such as invalid commands, policy violations in read-only mode, blocked root login, SSH authentication failure, unreachable server, SSH connectivity failure, LLM API failure, invalid cron request, or internal processing error.

The interface also offers troubleshooting tips like checking server connection, SSH credentials, LLM configuration, and application logs. This design guarantees honest and reliable delivery of validation and execution errors without revealing confidential information.

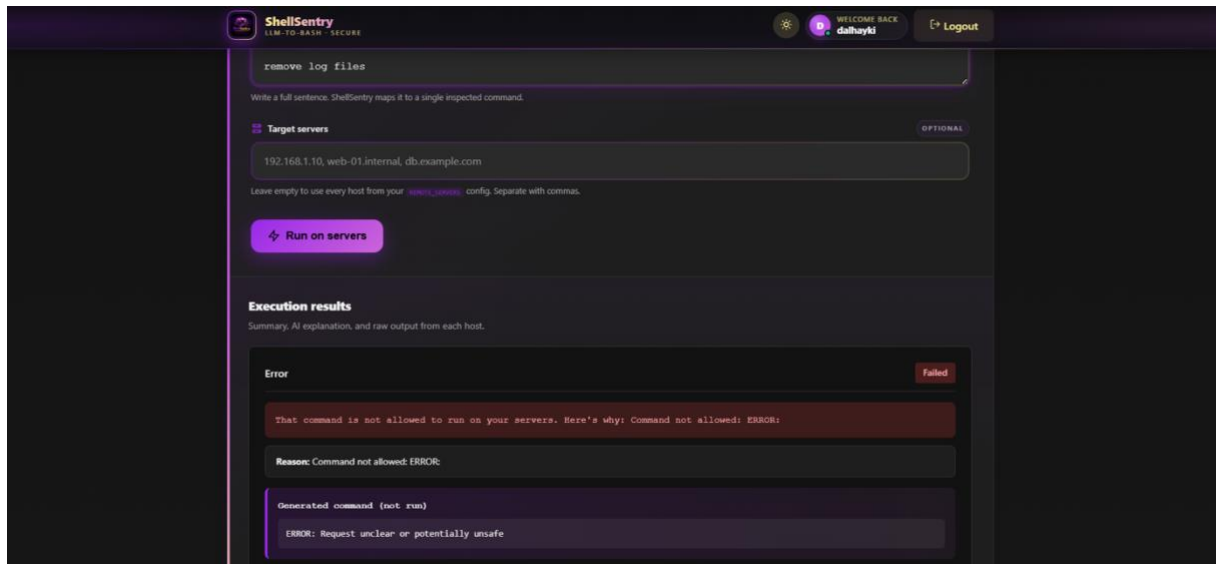


Figure 13: Error Handling

5.3.5 Navigation and session flow

ShellSentry controls access to protected functionality using a flow of navigating sessions. Unauthenticated users are taken to the landing, login, and/or registration pages, while authenticated users are given access to the dashboard and execution features. Once the user logs in, the user is redirected to the main dashboard, where they can submit requests through natural language and target servers. The navigation structure enables users to navigate between execution results and archived script operations, as well as to scripts and the Safe Cron management features, if available. Flask-Login is used to handle user sessions (Flask-Login Documentation, 2026); sensitive routes are protected, and unauthorized access to execution APIs is prevented. The logout function securely ends the active session and returns the user to the login page. The entire interaction and navigation process of the system is shown in Figure 14, which shows a comprehensive ShellSentry user flow from authentication to command execution and reporting.

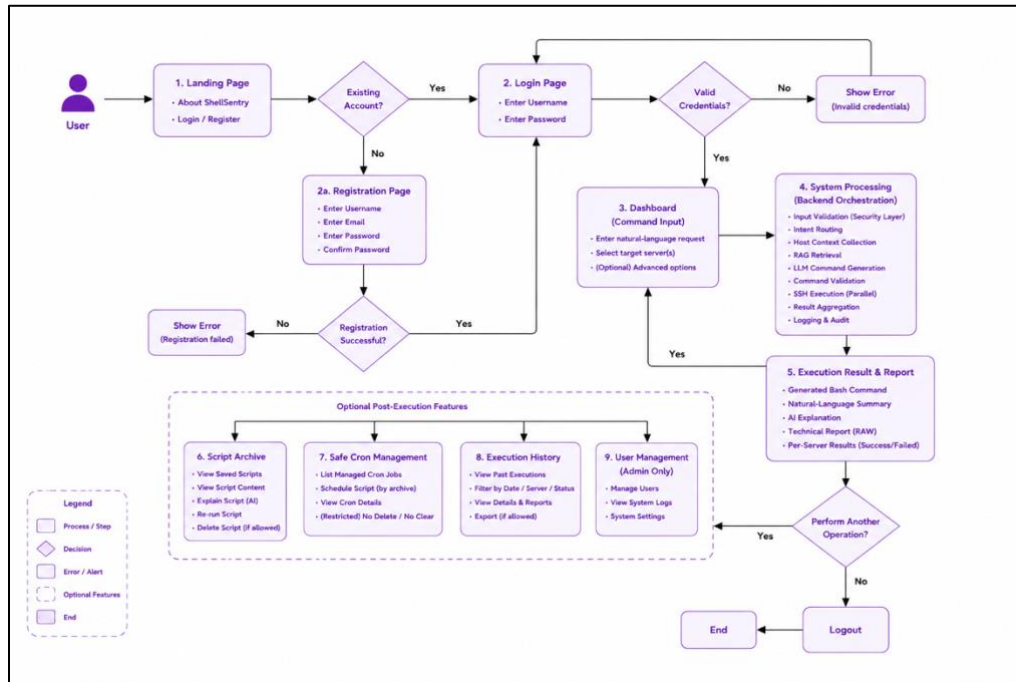


Figure 14: User flow

This figure depicts how the ShellSentry platform handles user authentication, command execution, orchestration of operations in the backend, and features relating to post-execution management. It starts with the landing page, where the user registers or logs in, then verifies the credentials and gains access to the dashboard to enter a server command. The back-end processing layer will process and validate security, route intents, retrieve RAG, generate AI commands, execute them through SSH, and aggregate results. The user is presented with detailed reports after execution, and has the option to use various features before logging out or executing further processes: script archiving, cron job management, execution history, and administrative user management.

5.4 Core Algorithms

This section introduces the basic algorithms implemented in ShellSentry. These algorithms manage secure natural-language processing, command validation, command execution, script management, and retrieval-augmented generation. They work together to offer deterministic security validation and operational traceability, along with controlled AI-assisted remote administration.

5.4.1 Natural-Language Execution Algorithm

This algorithm is the main execution pipeline of ShellSentry. It facilitates authentication, input checking, route resolution, host probing, retrieval-augmented generation, command validation, SSH execution, output formatting, and audit logging.

Pseudocode:

Algorithm: Natural-Language Execution

REQUIRE authenticated user

READ:

natural-language request
target servers


```

IF request is empty OR unsafe:
    RETURN validation error
IF no servers selected:
    use default configured servers
Detect execution route using AI and rule checks
IF route is cron operation:
    handle Safe Cron request
    RETURN result
IF route is archive operation:
    handle archived scripts
    RETURN result
Collect host context from target servers
Retrieve related RAG examples
Generate Bash command using AI
IF generated command is invalid:
    RETURN blocked-command response
Normalize validated command
Execute command on remote servers through SSH
Format execution results
Generate AI explanation and summaries
Store execution logs
RETURN:
    generated command
    execution results
    summaries
    explanations
IF unexpected error occurs:
    RETURN internal server error

```

5.4.2 Command Validation Algorithm

This algorithm checks for valid Bash commands before remote execution. Implements security policies such as blacklist checking, whitelisting, read-only policy enforcement and privileged operation controls.

Pseudocode:

```

Algorithm: Command Validation
READ:
    generated command
Remove markdown formatting and shebangs
Split command into segments
Check blacklist patterns
Check whitelist commands
Check restricted operations
IF read-only mode is enabled:
    block state-changing commands
IF root execution is disabled:
    block sudo and privileged operations
Normalize validated command

```

```
RETURN:
    valid OR blocked status
```

5.4.3 Parallel SSH Execution Algorithm

This algorithm is able to handle concurrent execution of commands from multiple Linux hosts via SSH connections while isolating host-level failures.

Pseudocode:

```
Algorithm: Parallel SSH Execution

READ:
    approved command
    target servers
FOR EACH server:
    verify SSH connectivity
PARALLEL EXECUTION:
    authenticate using configured credentials
    execute command through SSH
    collect stdout, stderr, and exit code
Store per-server results
Determine overall status:
    success
    partial success
    failed
RETURN execution results
```

5.4.4 Script Archive and Safe Cron Algorithm

This algorithm manages script archival, script reuse, and controlled cron scheduling.

Pseudocode:

```
Algorithm: Script Archive and Safe Cron

READ:
    archive or cron request
IF script is multi-line:
    save script in archive directory
IF request is list archived scripts:
    RETURN archived scripts
IF request is rerun script:
    validate archived script
    execute validated script
IF request is explain script:
    generate AI explanation
IF request is cron scheduling:
    validate cron expression
```

```

        create managed cron entry
    IF destructive cron removal detected:
        block operation
    RETURN operation result

```

5.4.5 Host Context Collection Algorithm

This algorithm collects information from the remote Linux hosts in order to aid in the generation of environment-aware commands.

Pseudocode:

```

Algorithm: Host Context Collection

READ:
    target servers
FOR EACH server:
    connect through SSH
    collect operating-system information
    detect running services
    inspect listening TCP/UDP sockets
Store collected host context
RETURN host context

```

5.4.6 RAG Retrieval Algorithm

This is a retrieval-augmented generation (RAG) algorithm based on semantic similarity search with a curated knowledge base for Bash. (Lewis, 2020)

Pseudocode:

```

Algorithm: RAG Retrieval

READ:
    natural-language request
Generate embedding vector
Perform semantic similarity search
Retrieve top-k related command examples
Format examples for prompt grounding
RETURN grounded context

```

5.5 Mapping to Requirements

This section illustrates the tracing of system requirements from Chapter 4 to the design elements given in Chapter 5. The section documents how the functional and non-functional requirements were mapped to specific architectural modules, interfaces and algorithms, with the intent to structure the proposed design to meet the project objectives and still remain secure, maintainable, usable and AI-assisted remote-execution controlled.

5.5.1 Functional Requirements Coverage

The functional requirements of Chapter 4 are matched by the modular design of Chapter 5: LLMClient is used for generating Bash code in the natural language (5.4.1), SSHExecutor.probe_host_context is

used for host-aware operation (5.4.5), and RagPipeline embedding and similarity search over the curated knowledge base is used for grounding RAG (5.4.6).

Deterministic safety is split in SecurityLayer and CommandValidator: on user text and on generated / archived script text - including on normalized text before executing the command remotely (5.4.2). Along with Config defaults and credential resolution (5.4.3; dashboard target entry in 5.3.2), SSHExecutor offers seamless authentication and multi-server orchestration and SSH execution. Safe Cron list/schedule with blocked destructive removal is the combination of script archive list, rerun, and explain paths, execution_router, routes/api.py and SSHExecutor (5.4.4).

Audit logging and reporting are a combination of ExecutionLog, executor logging, result_formatter, and a Summary (optional) in LLMClient (5.2, 5.3.5, 5.4.1), to show all of these together in the UI's JSON output. Config is used for configuring policies (read-only default, root toggle, archive/cron/LLM settings) along with validation (5.2.3, 5.4.1-5.4.2).

5.5.2 Non-Functional Requirements Coverage

Security and defense in depth is spread over the input validation, the intent routing, and the post-generation policy, and no single layer is responsible for security. ExecutionLog and structured log fields record the requester of an action, what was generated, which hosts were targeted, and what was returned by each host, which provides accountability and traceability.

The dashboard layout and the natural-language summaries, optional AI explanations, and expandable technical reports (5.3) ensure usability and explainability. Reliability under partial failure is secured using parallel SSH that provides isolated per-host results, such that when one host fails, the others are not prevented from working.

The separation of concerns between routes, the execution router, validators, the SSH executor, the LLM client, RAG, and formatting modules (5.1) allows for keeping each non-functional concern in a clearly visible portion of the design that can be verified in Chapter 6.

5.6 Chapter summary

ShellSentry's system design was introduced in Chapter 5 to connect the requirements of Chapter 4 with the implementation and evaluation of the system in Chapter 6. A defense-in-depth, modular architecture was described: Flask routes are for the HTTP and authentication; SecurityLayer and execution_router control user input and intent (Safe Cron and script-archive paths); RagPipeline and LLMClient provide grounded, host-aware generation in the standard path; CommandValidator is for enforcing a deterministic policy prior to any remote run; and SSHExecutor is for parallel SSH execution with per-host outcomes and configurable authentication.

Design of the database is intentionally small (User, ExecutionLog) to enable login and auditable execution logs whilst maintaining operational state on remote Linux hosts. The design includes interface elements such as landing, login, registration, command dashboard, rich result, and error reporting with optional AI explanations. In Section 5.4, the basic algorithms that are used to carry out all the functions were formalized. All the functions were carried out by using the basic algorithms, which were formalized in Section 5.4. To allow design decisions to be traceable, functional and non-functional requirements were mapped onto these components in section 5.5.

In general, the design focuses primarily on deterministic security validation, accountable remote execution, controlled automation with AI assistance, and reportable within the confines of an educational prototype.

Chapter 6

System Implementation and Testing

This chapter outlines the implementation of ShellSentry as a working prototype and the testing of the requirements and design presented in Chapters 4 and 5. The development environment, technology selection, project layout, configuration, and feature walk-through with screenshots are covered in Section 6.1. We now present the evaluation methodology, functional and security testing, performance observations, optional model comparisons, and a brief strengths and weaknesses analysis tied to Chapter 2, in Section 6.2.

6.1 System implementation

This section outlines the process of turning the design presented in Chapter 5 and the technology and software components selected in Chapter 4 into a working prototype and outlines how the components needed to be integrated into the selected deployment environment to meet the defined system requirements in Chapter 4. The implementation highlights secure generation of Bash with AI, authenticated remote execution, modularity, and auditable multi-host orchestration.

6.1.1 Development environment and hardware specifications

ShellSentry has been developed and tested on the Windows 11 Home workstation (version 23H2, OS build 22631.6199). The development machine is equipped with an 11th Gen Intel Core i5-1135G7 @ 2.40GHz CPU and 7.8 GB of RAM. Storage for development is on a system drive (C: total 405.85 GB, 69.3 GB free) and another drive (D: total 50.27 GB, 43.26 GB free). This prototype application was tested with Python 3.13.0 (via python run.py) and the service served via Flask service, both through the web dashboard and REST API.

The test environment consisted of Kali Linux and Raven Linux virtual machines running on Linux operating systems in the host machine, VirtualBox. By default, the target(s) in the prototype's .env file were 192.168.56.101, 192.168.56.102 and 192.168.56.104 – all these were accessed via SSH on port 22 and pre-flight reachability checks were made. The LLM integration was set up using an OpenAI-compatible endpoint (`LLM_API_BASE_URL=https://api.groq.com/openai/v1`) with the model llama-3.1-8b-instant, with the API key set in an environment variable (not in code).

6.1.2 Technology Stack Selection and Justification

Table 6 presents the main technologies and libraries used in ShellSentry, together with their implementation roles and justification.

Table 6: Main technologies and libraries used in ShellSentry

Technology / Tool	Version	Purpose in ShellSentry	Justification
Python	3	Main programming language	Provides strong library support for web development, AI integration, SSH communication, and rapid prototyping.
Flask	3.0.0	Web framework and REST API	Lightweight framework suitable for modular web applications and API routing (Flask Documentation, 2026).
Flask-Login	0.6.3	Session management and authentication	Protects execution routes and ensures only authenticated users can access system functionality (Flask-Login Documentation, 2026).
Flask-SQLAlchemy	3.1.1	ORM and database integration	Simplifies database operations and supports persistent audit logging (Flask-SQLAlchemy Documentation, 2026).
SQLite	Default	Prototype database	Lightweight database suitable for educational prototypes and local deployment (SQLite Documentation, 2026).
Werkzeug	3.0.1	Password hashing and verification	Improves credential security through hashed password storage (Werkzeug Documentation, 2026).
Requests	2.31.0	HTTP communication with LLM APIs	Supports OpenAI-compatible API communication and flexible provider integration.
Sentence-Transformers	Latest	Embedding generation for RAG	Generates semantic embeddings for similarity-based retrieval (Sentence Transformers Documentation, 2026).

all-MiniLM-L6-v2	Default model	Embedding model	Lightweight embedding model suitable for fast semantic search.
FAISS (faiss-cpu)	Latest	Similarity search engine	Performs efficient top-k retrieval over the command knowledge base (FAISS Documentation, 2026).
Paramiko	3.4.0	SSH communication and execution	Enables secure remote execution, host probing, and multi-server orchestration (Paramiko Documentation, 2026).
python-dotenv	Latest	Environment configuration loading	Loads SSH, LLM, and security settings from <code>.env</code> files without hardcoding secrets.
HTML / CSS / JavaScript	—	Frontend implementation	Provides the web interface, dashboard, result display, and client-side interaction.
Jinja2 Templates	Built-in with Flask	Server-rendered frontend pages	Simplifies integration between Flask backend and frontend pages.
openai	1.6.1	LLM-related dependency	Included in the project dependencies to support OpenAI ecosystem compatibility (LLM calls are implemented via OpenAI-compatible HTTP patterns) (OpenAI, 2026) (Groq, 2026).
bcrypt	4.1.2	Cryptography dependency (commonly for password hashing stacks)	Part of the dependency set for secure hashing; password hashing is handled through the Werkzeug hashing utilities in the prototype.

6.1.3 Project structure and module organization

The Flask server is started by `run.py`, and `src/app.py` loads the configuration and registers the routes. User pages (login, registration, dashboard) are done in `src/routes/pages.py`, and the main execution orchestration (POST `/api/execute`) is done in `src/routes/api.py`. The pipeline for the defense is distributed into `src/security.py` and `src/command_validator.py`, whereas the pipeline for AI generation and grounding is located in `src/llm_client.py` and `src/rag_pipeline.py`. The models for authentication and

audit logging are defined in `src/models.py`, while the remote probing and SSH execution are in `src/ssh_executor.py`. The UI is available from `templates/` and `static/` (JS/CSS), and audit data is kept on the client side in the SQLite database, `instance/`.

```
ShellSentry/
├─ run.py
├─ requirements.txt
├─ .env (local config/secrets; exclude from report upload)
├─ instance/
│   └─ shellsentry.db (SQLite audit DB)
├─ src/
│   ├── app.py
│   ├── routes/
│   │   ├── api.py
│   │   └─ pages.py
│   ├── execution_router.py
│   ├── services.py
│   ├── security.py
│   ├── intents.py
│   ├── command_validator.py
│   ├── ssh_executor.py
│   ├── llm_client.py
│   ├── rag_pipeline.py
│   ├── result_formatter.py
│   └─ models.py
├─ templates/
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   └─ errors/error.html
└─ static/
    ├── css/style.css
    └─ js/dashboard.js
```

Figure 15: Project tree structure

6.1.4 Configuration and deployment behavior

ShellSentry runs with `.env` configuration (`python-dotenv` and `src/config.py`). By default, it is based on the local SQLite database (`DATABASE_URL=sqlite:///shellsentry.db`), which can be used to store authentication data and the execution audit.

LLM integration is set to use an LLM API that is compatible with OpenAI and is called as a chat/completion endpoint through an HTTP call, with the LLM API type set to OpenAI, the LLM API base URL set to <https://api.groq.com/openai/v1>, and the LLM model selected to be llama-3.1-8b-instant. The behavior for remote SSH is activated via `SSH_USER=hero` and the list of targets to connect to in `REMOTE_SERVERS` (Raven Linux and Kali test hosts), and host-specific SSH overrides are

possible via `SERVER_CREDENTIALS` (which allows you to use different usernames/credentials for individual hosts without coding changes).

The deployment prototype launches the Flask service with `python run.py`, which runs the app in debug mode and binds to 0.0.0.0 on port 5001, for accessing the dashboard/API service. There are also configuration flags in `src/config.py` that control security and feature behavior. The `.env` file for development/testing had `READ_ONLY_EXECUTION` set to false and `ALLOW_ROOT_EXECUTION` set to true for more commands to be executed during implementation and demonstrations/readings, but deterministic safety was still respected in `SecurityLayer` and `CommandValidator`, structured into layers of input and command validation.

The prototype provides configuration support for the script archive and Safe Cron features through the following environment variables: `SCRIPT_ARCHIVE_DIR_NAME`, `SCRIPT_ARCHIVE_MAX_LIST`, `SAFE_CRON_MODE`, `SAFE_CRON_TAG_PREFIX`; the defaults for these are specified in `src/config.py` (e.g., enabled Safe Cron filtering, unlimited number of archived scripts, and a separate remote script archive directory).

6.1.5 Implemented features vs Chapter 5 design

ShellSentry is deployed in a modular fashion as described in Chapter 5. The web app and authentication flow (landing/login/register/protected) access/execute was achieved via Flask and Flask-Login, and the main execution orchestration was achieved in the `src/routes/api.py /api/execute`. The defense-in-depth routing logic (safe cron, script-archive action, or the standard NL→Bash flow) was done with the execution router (`src/execution_router.py`), with explicit policy blocks to block cron-removal intents.

Host-aware generation was added to `src/ssh_executor.py` using `probe_host_context` to retrieve per-host environment info over SSH. To provide top-k trusted examples to the LLM prompt, retrieval grounding was done in `src/rag_pipeline.py` by utilizing FAISS similarity search and sentence-transformer embeddings. The following security safety measures were added deterministically: `src/security.py` (input validation) and `src/command_validator.py` (command cleanup, blacklist checks, whitelist checks, read-only/privileged operation checks, and command normalization). The `src/ssh_executor.py` was used to remotely execute approved commands with a pre-flight check for each host's reachability and parallel execution per host, providing independent per-host results as well as an overall status.

The script-archive workflow (list, re-run, and explain previously saved scripts) and the Safe Cron scheduling (managed entry listing/scheduling and destructive removal blocking) have been added to the SSH executor layer, following the same design as all archive/cron operations are policy-gated and validated. Lastly, auditability was achieved by maintaining the persistent log of executions in a local database with each execution recorded as success, failed, or partial depending on the results of the execution on a per-host basis using the `ExecutionLog` model.

6.2 Evaluation and testing

This section describes the evaluation and testing of the prototype system, ShellSentry, to ensure that the system meets the functional, security, and performance requirements. Repeatable manual test scenarios similar to real usage conditions were used in the controlled laboratory environment detailed in Section 6.1. The evaluation consists of functional testing to ensure proper system operation, security testing to check the protection mechanisms, performance analysis of major operations, comparative testing of various LLM configurations, and comparisons with an existing real-world solution. The results obtained and their strengths and weaknesses are also discussed, as well as the relationship between the results obtained and the literature reviewed in Chapter 2.

6.2.1 Testing methodology

This testing was done in the laboratory environment detailed in Section 6.1 and is not a complete automated test suite. The main `/api/execute` behavior was checked for both single-host and multi-host runs, as was the use of default targets, and structured success/error responses were verified via functional checks. The results of security checks included a pass-fail result for session protection, early input rejection, post-generation command policy enforcement, and Safe Cron destructive-intent blocking. Waiting (TTFB) was used for measuring performance on successfully executed requests and was done across host count and in comparison with successful and rejected responses, where applicable. Lastly, a controlled comparison of the two models was made with the same prompt and host, switching the single model parameter `LLM_MODEL` on the same Groq-compatible API, with a median and mean TTFB of five times each for the five runs per model. These decisions emphasize the importance of "what happened" and "how the supervisor saw it" in providing evidence that is clear and easily reviewed by others, as well as recognizing some limitations, including limited sample size and provider/network variation.

6.2.2 Functional testing

Functional tests: confirmation that the behavior implemented matches the functional requirements of Chapter 4 and Chapter 5 design: authentication, `/api/execute` orchestration, host probing, RAG retrieval, generation of LLM, command validation, parallel SSH execution, script-archive, and Safe Cron branches (where applicable), and structured reporting.

Table 7: Representative functional test scenarios

ID	Scenario	Steps	Expected	Result	Notes
F1	Unauthenticated access	Open <code>/dashboard</code> without logging in	Redirect to login	Pass	Got 302
F2	Login	Valid credentials on <code>/login</code>	Redirect to dashboard	Pass	
F3	NL → Bash (read-only style query)	Enter benign inspection request; optional server list	200, success: true, per-host stdout / exit code	Pass	Example from logs: active connections → generated netstat - nlutp
F4	Default targets	Leave server field empty; <code>REMOTE_SERVERS</code> populated	Executes on configured defaults	Pass	Execute on all live server configured
F5	Multi-host partial outcome	Mix reachable and unreachable hosts	Reachable hosts return results; unreachable marked failed; overall may be partial	Pass	Logs showed port 22 not reachable on some hosts while another succeeded

F6	Validation rejection	Request unsafe command	400, natural_language_summary explains block; no execution on remotes	Pass	Example: “remove log files” led to validation failure So it prevent execution
F7	Archive list	Natural-language list archived scripts	List payload per host	Pass	
F8	Safe Cron list / schedule	List managed entries; schedule with valid .sh and cron	Managed behaviour only	Pass	
F9	Safe Cron destructive block	Phrasing implying crontab wipe	400, no destructive SSH	Pass	

The screenshot displays the ShellSentry login interface on the left, which includes a 'Sign in' header, a login prompt, and input fields for 'Username' and 'Password'. A 'Continue to dashboard' button is visible below the password field. On the right, the Chrome DevTools Network tab is open, showing a list of requests. The requests include 'dashboard' (302 status), 'login?next=%2Fdashboard' (200 status), 'style.css' (200 status), 'logo.webp' (200 status), and several CSS files with various status codes (200). The bottom of the Network tab shows summary statistics: '10 requests 754 kB transferred 767 kB resources Finish: 1.42 s DOMContentLoaded: 250 ms Load: 1.47 s'.

Figure 16: F1: Unauthenticated access

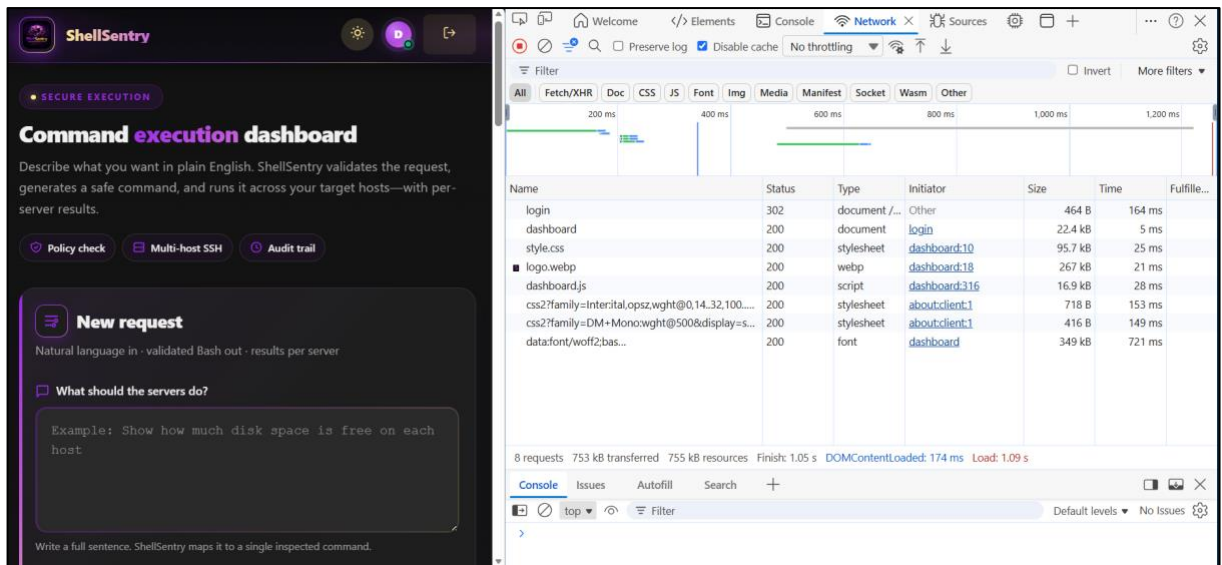


Figure 17: F2: Login

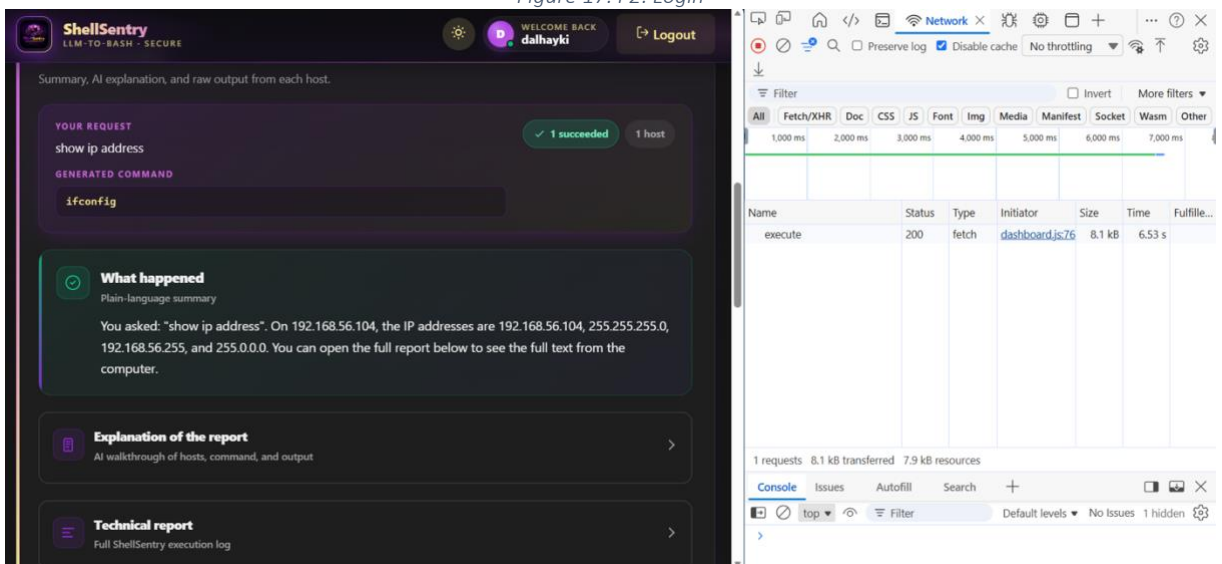


Figure 18: F3: NL → Bash (read-only style query)

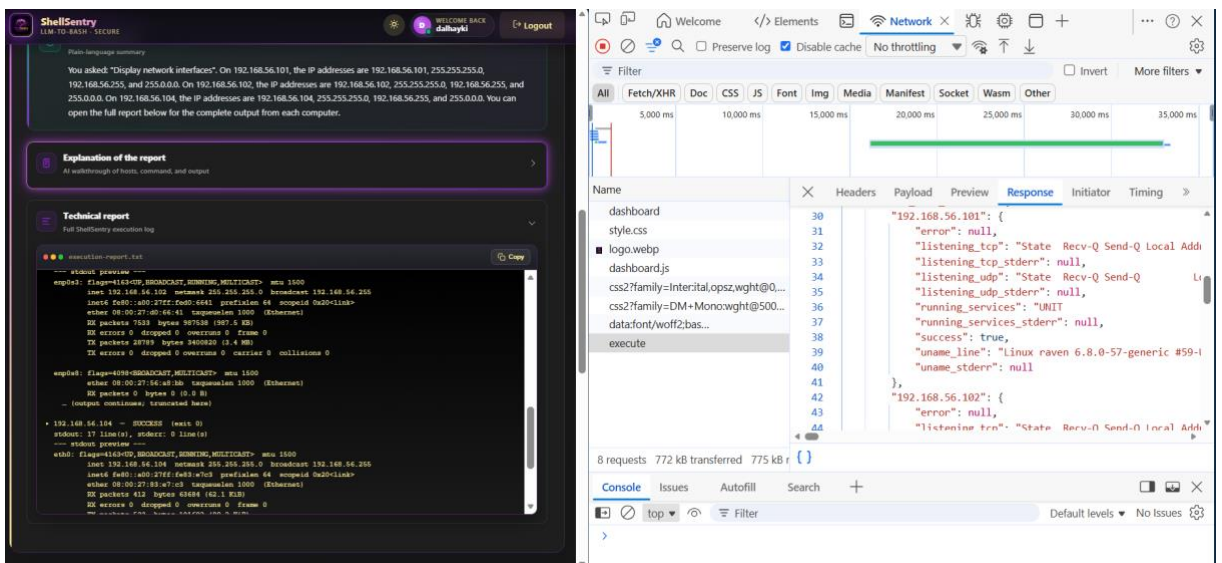


Figure 19: F4: Default targets

The ShellSentry interface shows a successful request for "show ip address". The generated command is "ifconfig". The "What happened" section provides a plain-language summary: "You asked: 'show ip address'. On 192.168.56.104, the IP addresses are 192.168.56.104, 255.255.255.0, 192.168.56.255, and 255.0.0.0. You can open the full report below for the complete output from each computer. 192.168.56.101: The computer did not answer in time. It may be busy or unreachable. 192.168.56.102: The computer did not answer in time. It may be busy or unreachable."

The network tab shows two requests:

Name	Status	Type	Initiator	Size	Time	Fulfil...
execute	200	fetch	dashboard.js76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js76	10.6 kB	12.72 s	

Figure 20: F5: Multi-host partial outcome

The ShellSentry interface shows an error: "That command is not allowed to run on your servers. Here's why: Command not allowed: ERROR:". The reason is "Command not allowed: ERROR:". The generated command (not run) is "ERROR: Request unclear or potentially unsafe".

The network tab shows four requests:

Name	Status	Type	Initiator	Size	Time	Fulfil...
execute	200	fetch	dashboard.js76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js76	10.6 kB	12.72 s	
execute	200	fetch	dashboard.js76	3.4 kB	6.25 s	
execute	400	fetch	dashboard.js76	471 B	7.13 s	

Figure 21: F6: Validation rejection

The ShellSentry interface shows a technical report for a validation rejection. The report details the execution of the command "ifconfig" on three hosts:

- 192.168.56.101: Server 192.168.56.101 is unreachable on port 22 within 5s. The host may be offline, a firewall may be blocking SSH, or the SSH service is not running.
- 192.168.56.102: FAILED (exit -1). Error: Connection timeout: 192.168.56.102 is unreachable.
- 192.168.56.104: SUCCESS (exit 0). Stdout: 1 line(s), stderr: 1 line(s).

The network tab shows three requests:

Name	Status	Type	Initiator	Size	Time	Fulfil...
execute	200	fetch	dashboard.js76	8.1 kB	6.53 s	
execute	200	fetch	dashboard.js76	10.6 kB	12.72 s	
execute	200	fetch	dashboard.js76	3.4 kB	6.25 s	

Figure 22: F7: Archive list

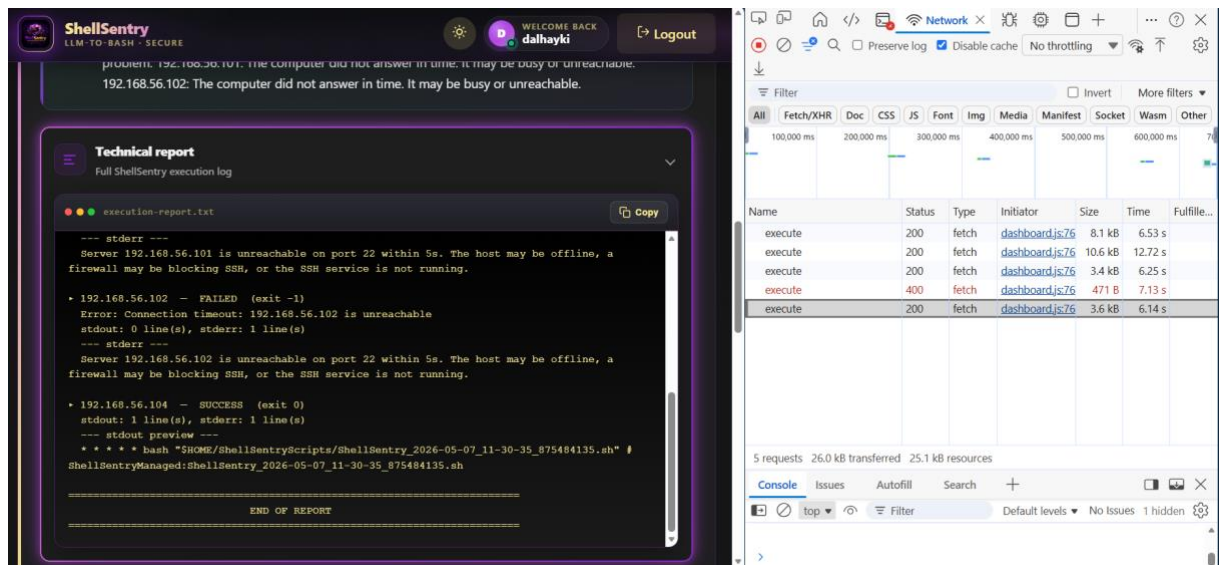


Figure 23: F8: Safe Cron list / schedule

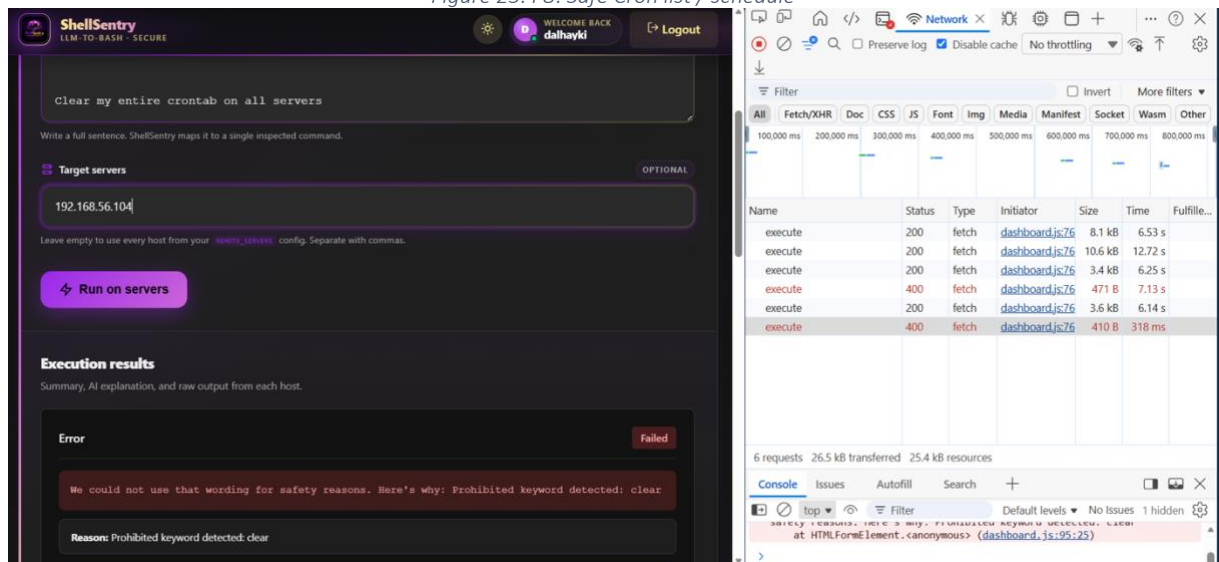


Figure 24: F9: Safe Cron destructive block

6.2.3 Security testing

Security testing was done on the controls that have been put in place in ShellSentry, and not on third-party penetration tests of unrelated infrastructure. The objective was to demonstrate that there are paths that are unsafe and sensitive configurations that are not in the source.

Table 8: Security-oriented checks

ID	Security objective	Test procedure	Expected outcome	Result
S1	Authenticated execution	While logged out, call POST /api/execute (DevTools fetch or direct tool)	302 or equivalent unauthenticated rejection	pass

		without session cookies		
S2	Session-only dashboard	Open /dashboard without login	Redirect to login	pass
S3	SecurityLayer input rejection	Submit natural language containing a prohibited keyword / pattern from security.py (use a harmless test string designed to trigger the rule)	400 with validation message; no SSH execution	pass
S4	Post-LLM deterministic block	Submit NL that yields a non-whitelisted or blacklisted command (or known-bad operator phrasing)	400; optional display of generated command (not run); no successful remote run	pass
S5	Read-only enforcement (<i>if <code>READ_ONLY_EXECUTION=true</code></i>)	Attempt an NL task that implies state change (package install, file delete, service restart)	Blocked by CommandValidator / read-only checks	pass
S6	Privileged execution gate (<i>if <code>ALLOW_ROOT_EXECUTION=false</code></i>)	Prompt that tends to produce sudo / root-style commands	Blocked when policy forbids	pass
S7	Safe Cron anti-wipe	Submit NL clearly requesting crontab clear/remove/wipe for all entries	400 “cron removal blocked” style response; no destructive cron SSH	pass
S8	Script archive re-validation (<i>if used</i>)	Re-run a saved .sh whose content would be invalid under current policy	400 before execution	pass
S9	XSS / injection in UI rendering	Use prompts / outputs containing <code><script>...</script></code> or HTML-like fragments	Dashboard shows escaped text (no script execution)	pass
S10	Secret hygiene	Confirm .env is listed in .gitignore	Secrets not committed	pass

		and not present in Git history for submission		
S1 1	Transport / deployment caveat (documentation test)	Record that prototype uses HTTP on localhost / LAN without TLS	Document as limitation for production	pass

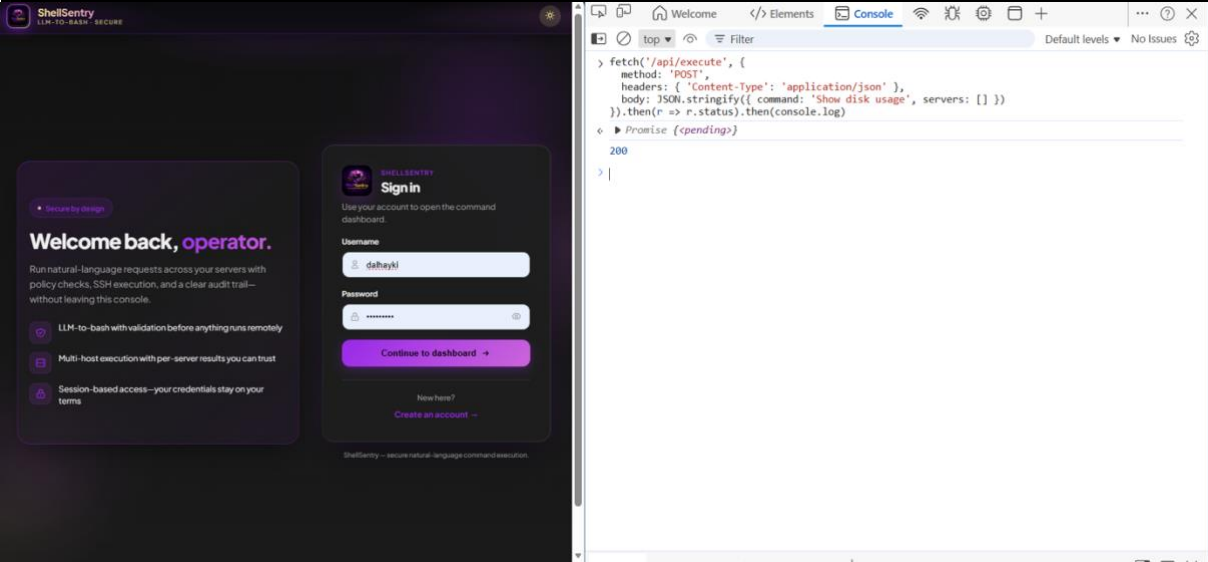


Figure 25: S1 : Authenticated execution(P1)

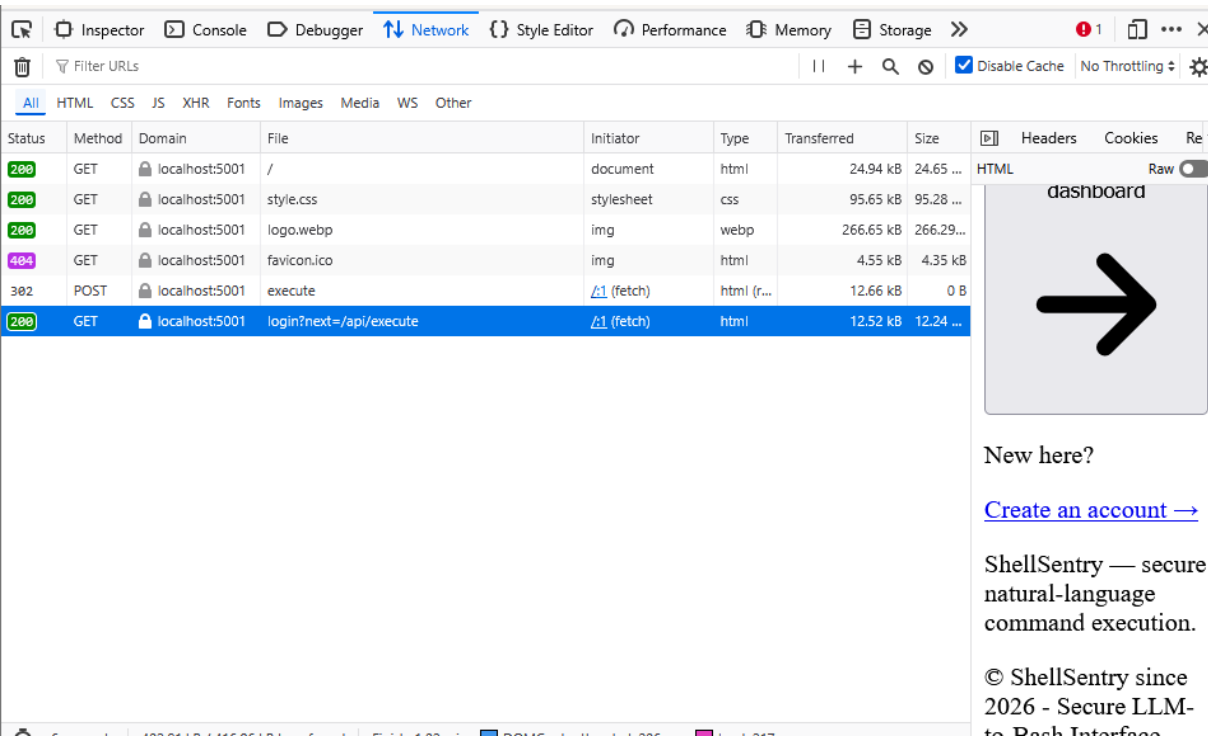


Figure 26: S1: Authenticated execution(P2)

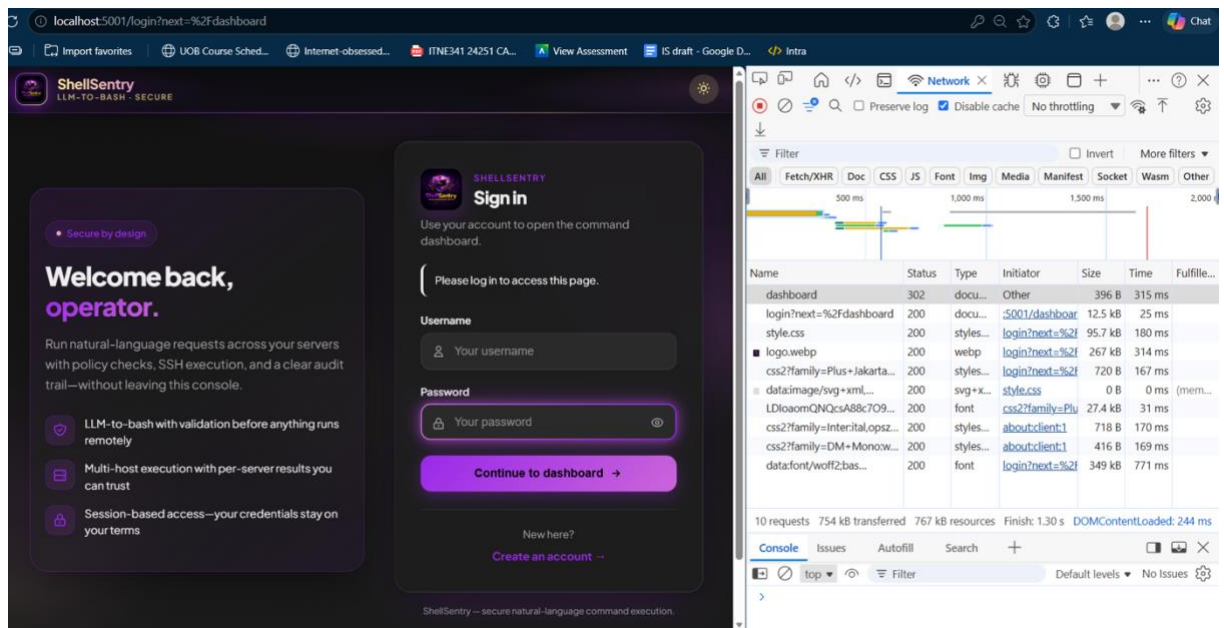


Figure 27: S2: Session-only dashboard

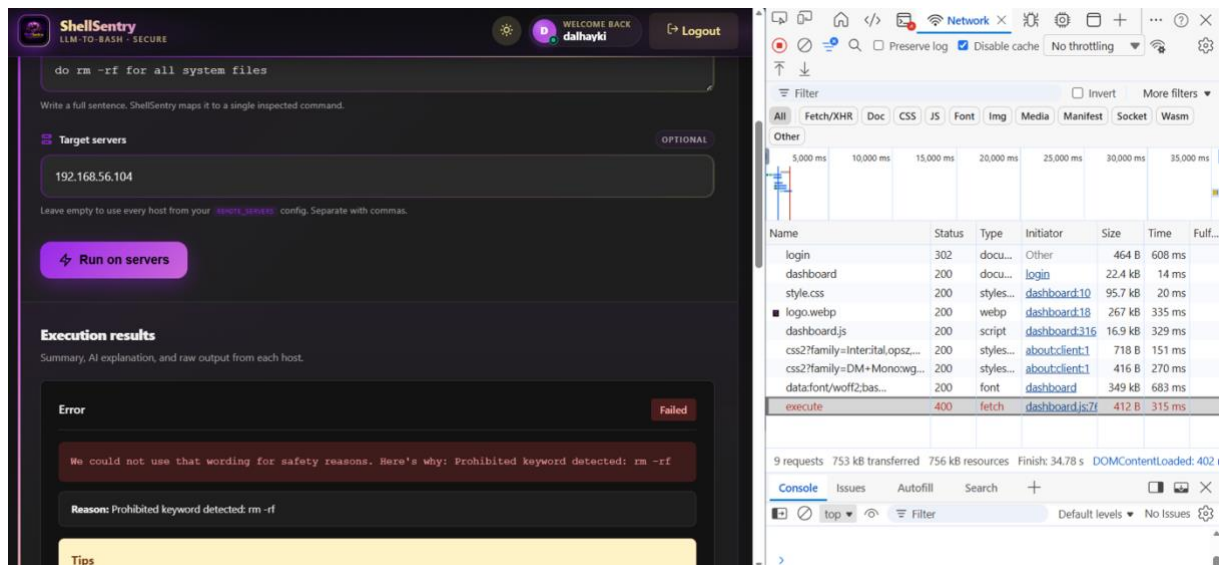
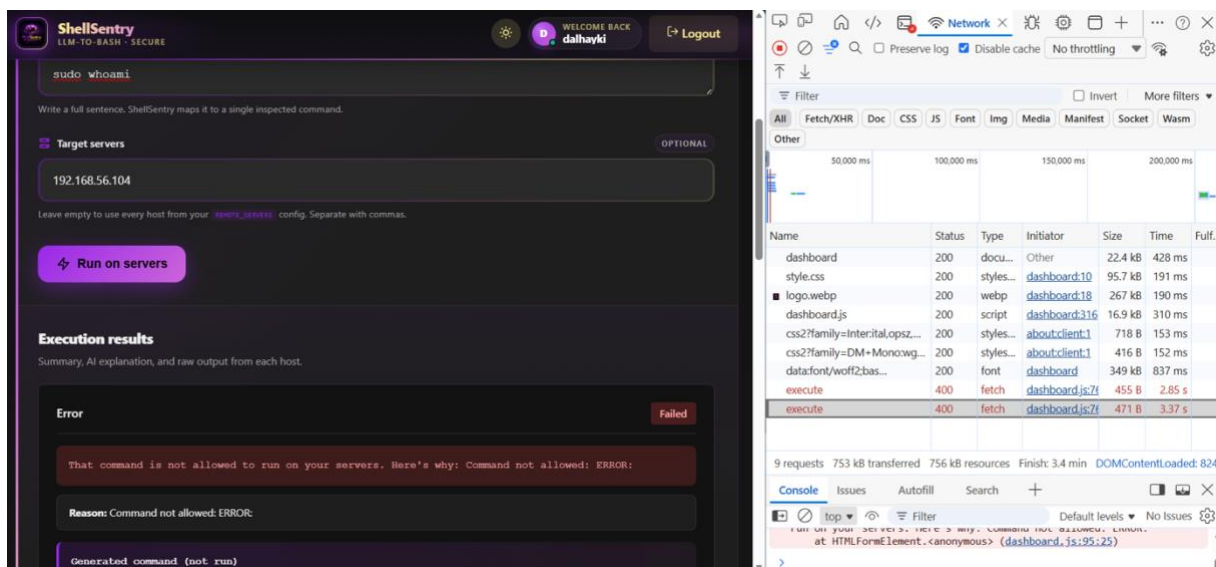
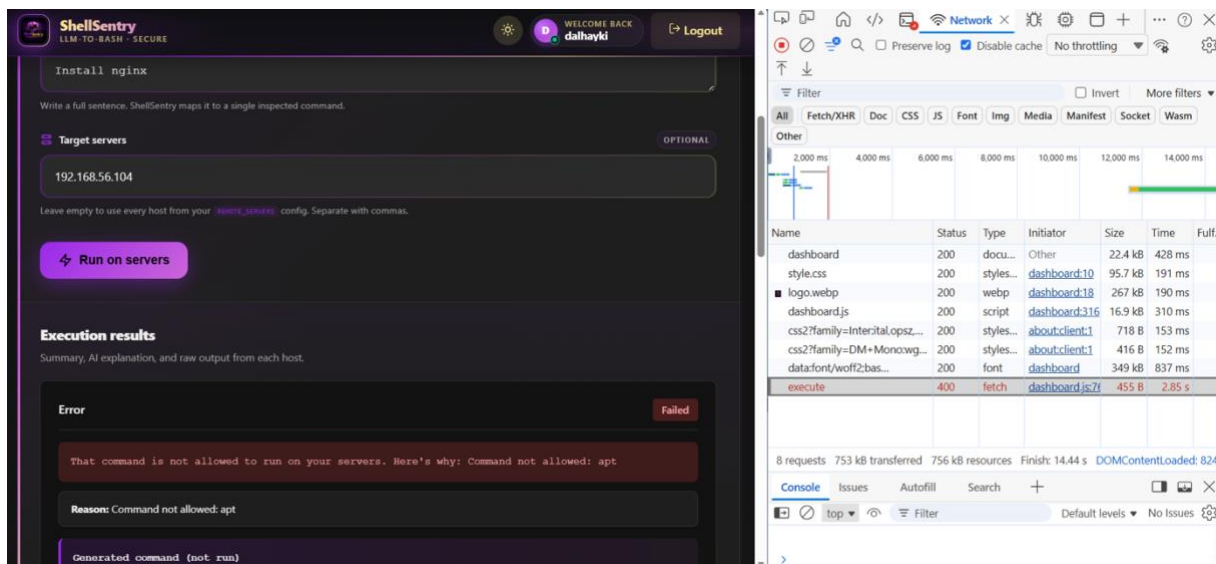
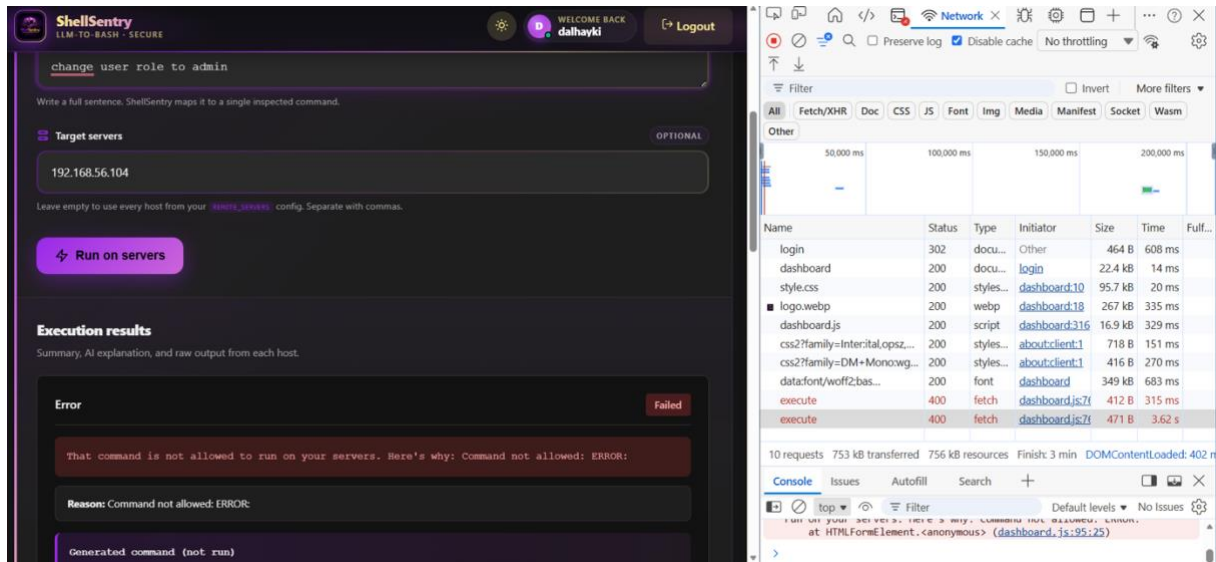


Figure 28: S3: SecurityLayer input rejection



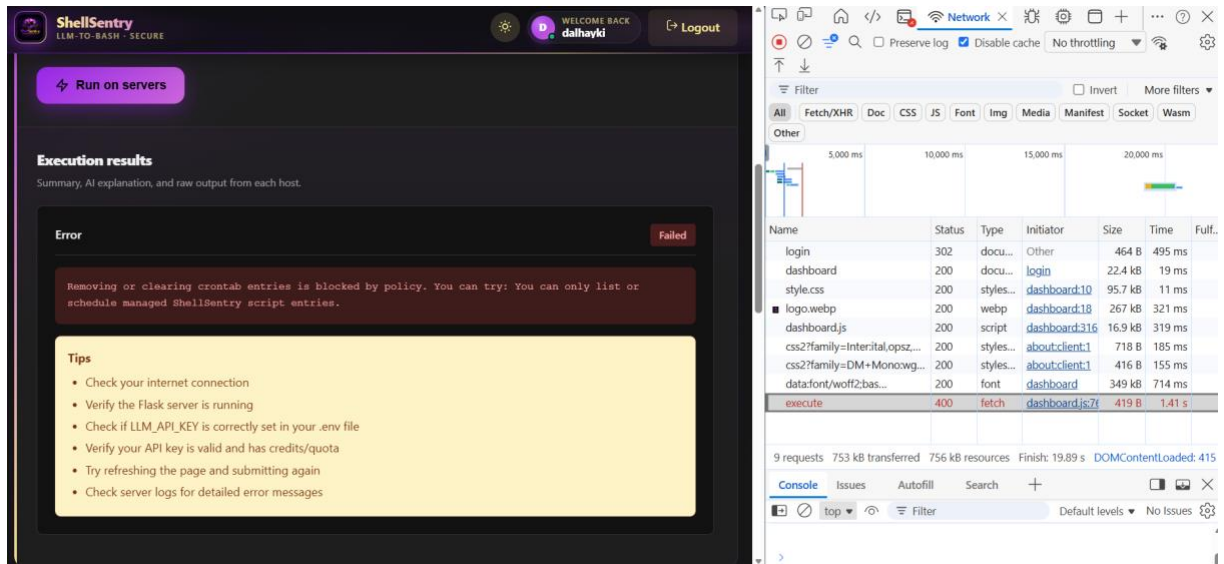


Figure 32: S7: Safe Cron anti-wipe

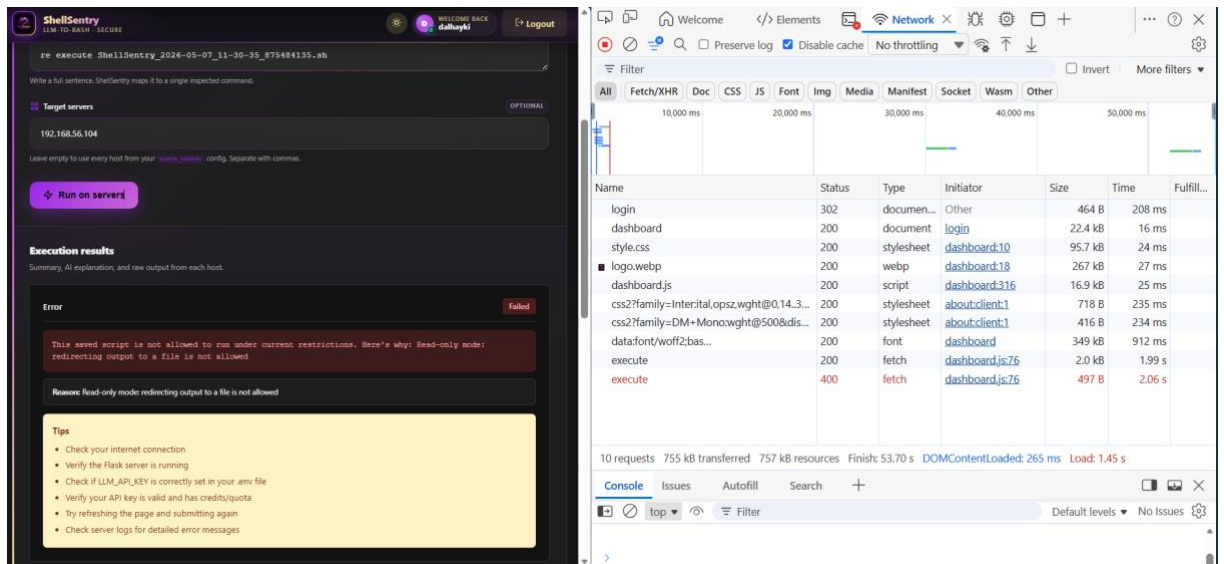


Figure 33: S8: Script archive re-validation

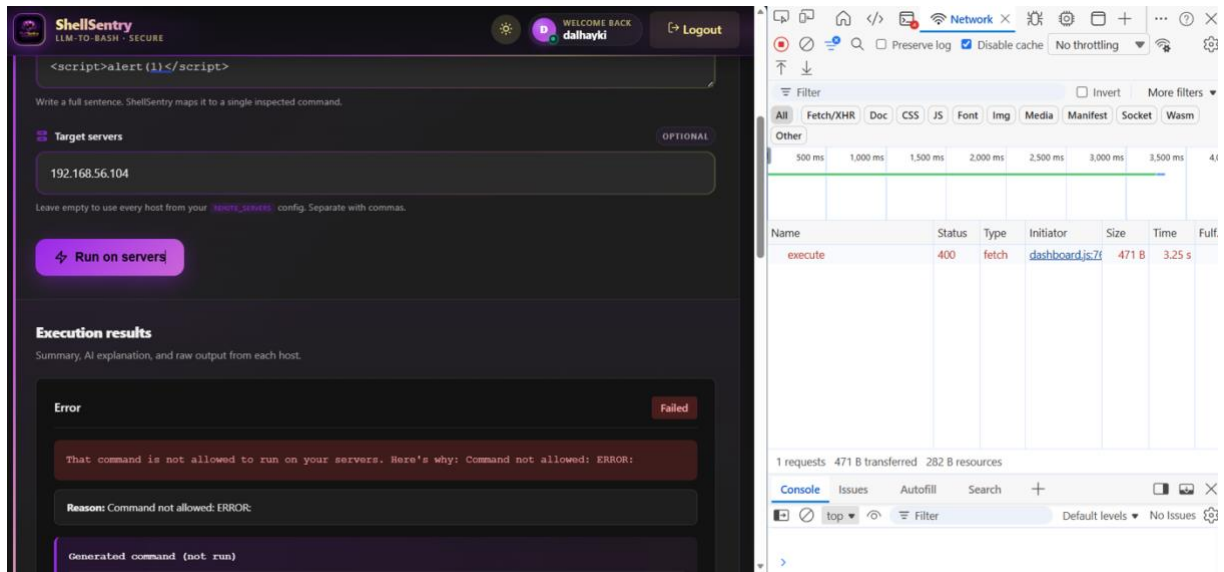


Figure 34: S9: XSS / injection in UI rendering

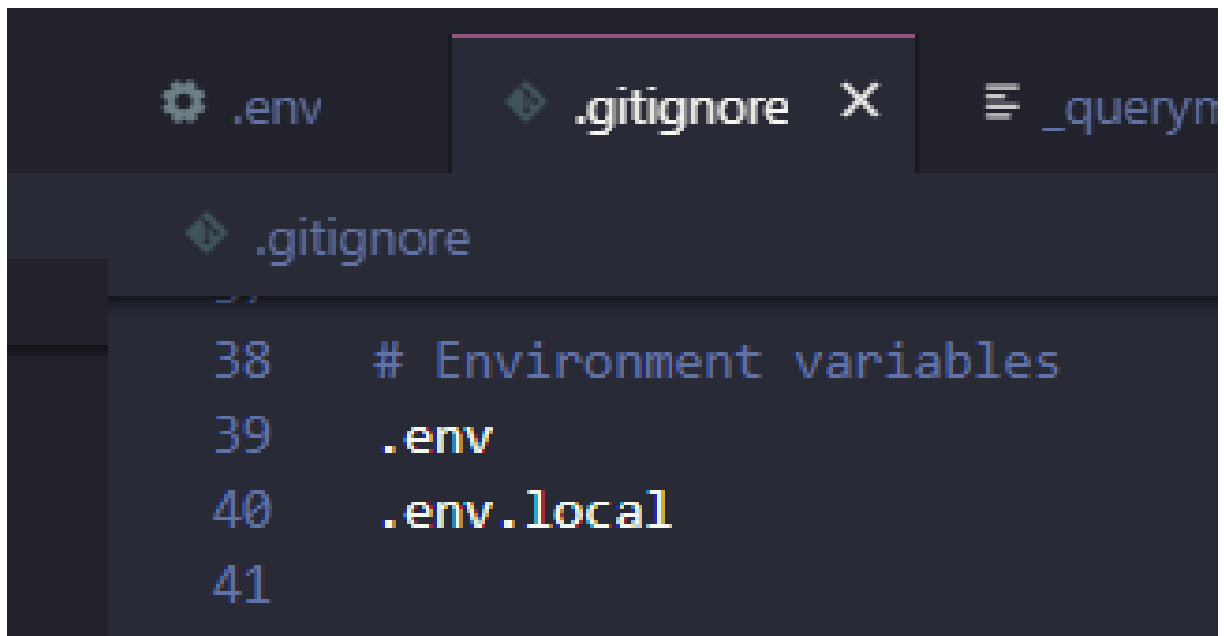


Figure 35: S10: Secret hygiene

6.2.4 Performance measurement

The performance was evaluated on the development workstation of Section 6.1.2, with Mozilla Firefox Developer Tools (Network panel). The POST /execute entry was chosen for each run, and the Timings tab was used to read the Waiting (TTFB) time, which is the time from when the request is sent until the first byte of the response is received. This value is considered a realistic measurement of the processing time on the server side for the ShellSentry pipeline (routing/probing/retrieval/LLM calls/validation/SSH execution/response assembly), but also contains a small part of the network/browser overhead. Very large Queued values (sometimes shown in minutes) were not interpreted as request latency. These figures are dominated by the duration of the request row in the Network timeline compared to the entire recording session, as multiple tests may be performed over the same duration. The analysis is based on Waiting (TTFB) instead of Queued duration, therefore. In these measurements, the LLM used was llama-3.1-8b-instant, which is accessed via the OpenAI-compatible HTTP endpoint specified in the project environment (such as [Grok-compatible LLM_API_BASE_URL](#) in [.env](#)).

Table 9: Observed timing (replace with your measurements)

Test ID	Scenario	HTTP outcome	Runs (n)	TTFB (Waiting): min–max	Mean (s)	Median (s)	Notes
P1	Three target hosts	200	5	4.29 – 45.22	≈ 23.10	≈ 12.81	Tail latency on some runs (~45 s).
P2	One target host	200	5	1.66 – 2.78	≈ 2.03	≈ 1.82	Stable “single-host” benchmark set.
P3a	Successful execution (paired with P3b)	200	2	2.98 – 3.65	≈ 3.32	≈ 3.32	For two samples, mean and median coincide with the midpoint $(2.98+3.65)/2$.
P3b	Rejected execution	400	2	≈ 1.51 – 1.51	≈ 1.51	≈ 1.51	Small error JSON; both failures ~1.51 s TTFB.

6.2.5 LLM comparison (Groq API models)

Initially, we found that using locally hosted large language models (such as Ollama or other runtime environments) would allow us to keep all the inference locally within the machine, and thus minimize reliance on external APIs. When running the same workstation (Intel Core i5-1135G7, 7.8 GB RAM) to perform a comparable open-weight model, without any significant slowdowns or instability during interactive testing, the existing prototype stack (Flask, sentence-transformers embeddings, FAISS, and parallel SSH workloads) did not leave enough headroom for development. Therefore, the comparative evaluation was performed with Groq's OpenAI-compatible hosted API to enable model switching by modifying the `LLM_MODEL` and maintaining the same `LLM_API_BASE_URL` and credentials.

A controlled repeatability setup was used in the comparison; the same natural-language prompt and the same single target host (192.168.56.104) were used for each model configuration. Five successful HTTP 200 executions were recorded for end-to-end latency (Waiting (TTFB)) from Firefox → Network → POST /execute → Timings. This is not just a metric for LLM inference alone, but also an all-around server-side workload (which includes LLM inference and remote execution).

Table 10: Two-model comparison under identical prompt/host conditions

LLM_MODEL	n	Median TTFB (s)	Mean TTFB (s)	Notes
llama-3.1-8b-instant	10	4.18	4.67	Same prompt/host; five successful runs
llama-3.3-70b-versatile	10	4.97	7.97	Same prompt/host; five successful runs

```
# LLM API Configuration
LLM_API_TYPE=openai
LLM_API_KEY=xxxxxxx
LLM_API_BASE_URL=https://api.groq.com/openai/v1
LLM_MODEL=llama-3.3-70b-versatile
```

Figure 36: LLM Versatile Configuration

```
# LLM API Configuration
LLM_API_TYPE=openai
LLM_API_KEY=xxxxxxx
LLM_API_BASE_URL=https://api.groq.com/openai/v1
LLM_MODEL=llama-3.1-8b-instant
```

Figure 37: LLM Instant Configuration

6.2.5.1 Comparison of Findings

The results for the other three models showed that llama-3.1-8b-instant had a lower median TTFB (4.18 s) than llama-3.3-70b-versatile (4.97 s), suggesting marginally improved performance for the typical successful runs in this sample with the smaller model configuration. The mean (7.97 s) of the larger model is greater than the mean (4.67 s) of the smaller model; however, because one run in the five-sample set had a much longer tail latency than the rest, this value was higher for the larger model than the smaller. For the task, both models generated acceptable command variants (the majority ss - tlnp, and one netstat -tlnp for the 8B model), concordant with the variance of LLM sampling for this controlled check, instead of some operational error.

6.2.5.2 Comparison Limitations

It is a small-sample (five runs per model) comparison based on the end-to-end latency as observed by the browser, which includes the time required to split the inference time between SSH and formatting. Also, there is the added variation of the network and provider side of the hosted inference, which would be reduced by local inference, but was not possible on the hardware available for this prototype scope.

6.2.6 Comparison with a Real-World Solution

ShellSentry can be likened to Shell-GPT, an AI-driven terminal assistant that translates natural language queries into shell commands. Both systems are seeking to make command-line interaction easier, with the help of Large Language Models (LLMs). But, ShellSentry has a different emphasis: secure remote orchestration, deterministic validation, and controlled execution of multiple Linux servers.

Shell-GPT's primary function is as a user-friendly command-line tool for local use. ShellSentry offers a web-based interface with authenticated access, multi-server SSH execution, host-aware generation, script archival, Safe Cron scheduling, and execution audit logging, to name just a few features. Further, ShellSentry adds retrieval-augmented generation (RAG), deterministic command validation, read-only execution policies, and root-execution restrictions to mitigate risks from AI-generated shell commands.

While ShellSentry offers enhanced security and orchestration capabilities, Shell-GPT's terminal integration and minimal CLI interaction are not part of the prototype at this time. ShellSentry is more about controlling remote administration and security-related workflows than productivity in the individual terminal (TheR1D, 2026).

Table 11: presents a comparison between ShellSentry and Shell-GPT.

Feature	ShellSentry	Shell-GPT
Natural-language-to-Bash generation	Supported	Supported
Web-based dashboard	Supported	Not supported
Direct CLI integration	Not supported	Supported
Multi-server SSH execution	Supported	Limited
Host-aware context collection	Supported	Limited
Retrieval-augmented generation (RAG)	Supported	Not supported
Deterministic command validation	Supported	Limited
Read-only execution mode	Supported	Not supported
Root-execution restriction	Supported	Not supported
Script archival and Safe Cron	Supported	Not supported
Audit logging and reporting	Supported	Limited
AI-generated execution explanations	Supported	Limited

While ShellSentry is being used in a prototype as an educational tool, the comparison shows the potential for moving beyond the current AI terminal assistants and adding features like orchestration, validation, auditability, and controlled automation that address security concerns.

6.2.7 Strengths and Weaknesses

ShellSentry is key because it is modular, has multiple layers of security validation, generates host-aware AI, runs multiple SSH processes in parallel, logs audits, and reports, explaining the process. It has drawbacks such as the fact that LLM outputs are probabilistic, security policies limit flexibility, it doesn't support advanced features for enterprises, and it only works in a controlled lab environment.

6.2.7.1 Strengths

ShellSentry has some good architecture and security attributes. It employs a modular pipeline architecture, where the different stages of routing, validation, AI generation, SSH execution, and reporting are executed in separate modules, enhancing the maintainability and scalability of the system. All security is done in layers, with layered input validation, intent routing, deterministic command validation, and execution restrictions.

Furthermore, host-aware prompting and retrieval-augmented generation (RAG) enhance the relevance and consistency of AI-generated Bash commands by leveraging trusted examples and real-time host context.

It also enables SSH execution in parallel with reachability pre-checks, providing an efficient way to orchestrate multiple servers without introducing host-level failures. Moreover, the ExecutionLog model ensures the auditability of requests, commands generated, target servers, and execution results. Both technical and more common users find it easier to use, thanks to the human-readable summaries and AI-generated explanations.

6.2.7.2 Weaknesses

Despite these strengths, the prototype has several limitations. Since command generation relies on a Large Language Model, the generated output remains probabilistic and may still require strict validation and human oversight. The whitelist and restricted-command policies also introduce a trade-off between security and operational flexibility.

Additionally, the current prototype does not implement advanced enterprise features such as full role-based access control (RBAC), immutable SIEM-integrated audit logs, human-in-the-loop approval workflows, or command dry-run simulation. The evaluation environment is limited to a controlled lab setup and primarily English-language interactions. Finally, system performance may vary depending on external LLM API response times, provider rate limits, and virtual-machine resource availability.

6.2.8 Discussion and Comparison with Literature

The evaluation results are very consistent with the results mentioned in Chapter 2, which are related to limitations and risks of using Natural Language to Bash (NL2SH) systems. Other research works like ScriptSmith and LLM-as-a-Judge pointed out that the generation of bash scripts is not enough if there are no validation and refinement mechanisms. These principles are embodied in the execution pipeline of ShellSentry, command validation and execution determinism, audit logging, and constrained SSH execution.

In addition, the testing phase revealed in the literature review that security issues exist, such as the risk of hallucinated or destructive AI-generated commands. Whitelist, blacklist, read-only, and root-execution restrictions were able to prevent unsafe commands from being executed on remote hosts, similar to what was observed in the translation study on NL2SH.

Moreover, the literature survey in Chapter 2 did not cover secure multi-host orchestration, authenticated remote execution, audit logging, and controlled scheduling extensively. ShellSentry's ability to go beyond simply command generation capabilities is that it also allows for host-aware prompting, retrieval-augmented grounding, parallel SSH execution, script archival, and Safe Cron scheduling within a single security-focused platform.

6.3 Chapter Summary

In this chapter, the implementation and evaluation of the ShellSentry prototype were presented to show how the ShellSentry system design described in Chapter 5 was realized in a working secure AI-assisted remote administration platform. These features included authenticated web access, generation of Bash

scripts by LLM, retrieval-augmented grounding, deterministic command validation, SSH-based multi-server execution, script archival, and Safe Cron management—features that were integrated into the implementation.

Functional testing, security validation, performance measurements, and comparative analysis were conducted, and the results confirmed that the prototype achieved its essential functional, security, and performance goals.

The findings validated the effectiveness of the layered security controls, the parallel remote execution model, and the audit logging technique, and also pointed out some prototype limitations, including LLM probabilistic behavior, evaluation limitations for prototypes, and reliance on external API performance.

Chapter 7

Conclusion and Future Work

This project has been designed and developed to create a secure web-based prototype named ShellSentry that transforms natural language queries into validated bash scripts and sends the scripts to remote Linux hosts for execution. The system was designed to overcome the dangers of executing commands by hand and LLM-generated shell commands by supporting authenticated access, input validation, retrieval-augmented grounding, host-aware prompting, deterministic command validation, and SSH-based execution. Script archiving, Safe Cron scheduling, audit logging, and human-readable reporting were some of the other features.

The implementation demonstrated that the automation of shells with the help of AI can be made more secure if the LLM is considered an untrusted entity, instead of an execution authority. ShellSentry validates generated commands before running them, captures execution information with ExecutionLog, and provides structured output to enable both technical and non-technical users. The system meets the essential functional and security requirements, such as authenticated execution, unsafe-command blocking, multi-host SSH execution, script archive workflows, and Safe Cron restrictions.

In sum, ShellSentry shows that natural-language-to-Bash systems need not be about commands alone. Rather, layered validation, traceability, least-privilege assumptions, and clearly communicating with the user about the result are the keys to secure execution.

7.1 Project Limitations

While ShellSentry met its primary goals, there are some shortcomings in the prototype. The system was tested in a lab setting with virtual Linux hosts, and additional testing would need to be done prior to deployment in a real production network setting. The prototype relies on an external provider of an OpenAI-compatible LLM API, which means that the availability and time since the first external provider might be subject to rate limitations and performance fluctuations.

Also, ShellSentry does not support complete role-based access control, immutable SIEM-integrated audit logs, human approval workflows, or dry-run simulation. The system is primarily designed to handle natural-language input in English and support operation on the target systems using SSH and other operating systems. Remote management methods are not in scope at this time.

7.2 Future Work

There are several directions for the improvement of ShellSentry in the future. Firstly, role-based access control could be introduced for different users to have varying command permissions. Secondly, a human-in-the-loop (HITL) approval workflow could be used for sensitive or state-changing commands. Third, a dry-run or simulation mode might give users the ability to see how command effects will be performed before they are actually used. Fourth, logs could be enhanced with immutable logs and integration with SIEM.

Future enhancements can include improved script integrity checks with hashing or signatures, wider input language support, more sophisticated RAG knowledge bases, automation target support, and deployment production features like TLS, secret management, and more stringent policy profiles.

7.3 Chapter Summary

This chapter wrapped up the ShellSentry project by providing a summary of the work done, the key findings, existing challenges, and potential future improvements.

Overall, the project successfully showcased a secure and accountable method of automating Bash operations using AI and highlighted the fact that there are additional areas for improvement in the future to ensure that such a system can be deployed in high-risk production environments.

References

- Flask Documentation, 2026. *Flask Documentation*. [Online]
Available at: <https://flask.palletsprojects.com>
[Accessed 15 Dec 2025].
- Flask-Login Documentation, 2026. *Flask-Login Documentation*. [Online]
Available at: <https://flask-login.readthedocs.io/>
[Accessed 26 nov 2025].
- Flask-SQLAlchemy Documentation, 2026. *Flask-SQLAlchemy Documentation*. [Online]
Available at: <https://flask-sqlalchemy.palletsprojects.com/>
[Accessed 12 jan 2026].
- Paramiko Documentation, 2026. *Paramiko Documentation*. [Online]
Available at: <https://www.paramiko.org/>
[Accessed 12 jan 2026].
- Sentence Transformers Documentation, 2026. *Sentence Transformers Documentation*. [Online]
Available at: <https://www.sbert.net/>
[Accessed 13 jan 2026].
- FAISS Documentation, 2026. *FAISS Documentation*. [Online]
Available at: <https://faiss.ai/>
[Accessed 20 jan 2026].
- SQLite Documentation, 2026. *SQLite Documentation*. [Online]
Available at: <https://www.sqlite.org/>
[Accessed 21 jan 2026].
- Werkzeug Documentation, 2026. *Werkzeug Documentation*. [Online]
Available at: <https://werkzeug.palletsprojects.com/>
[Accessed 27 jan 2026].
- Groq, 2026. *Groq API Documentation*. [Online]
Available at: <https://console.groq.com/docs>
[Accessed 28 jan 2026].
- OpenAI, 2026. *OpenAI API Documentation*. [Online]
Available at: <https://platform.openai.com/docs>
[Accessed 1 feb 2026].
- OWASP Foundation, 2026. *OWASP Top 10 Web Application Security Risks*. [Online]
Available at: <https://owasp.org/>
[Accessed 1 feb 2027].
- TheR1D, 2026. *Shell-GPT GitHub Repository*. [Online]
Available at: https://github.com/TheR1D/shell_gpt
[Accessed 5 feb 2026].
- Chen, Y. Z. X. W. H. L. J., 2024. *ScriptSmith: Automated Bash Script Generation, Assessment, and Refinement for IT Operations*. [Online]
Available at: <https://arxiv.org/abs/2403.03892>
[Accessed 10 feb 2026].

Yao, S. Z. J. Y. D. W. X. T., 2024. *LLM-Supported Natural Language to Bash Translation: Functional Evaluation and Reliability Analysis*. [Online]
Available at: <https://arxiv.org/abs/2402.11382>
[Accessed 15 feb 2026].

Zhang, H. L. Q. C. M., 2024. *LLM-as-a-Judge for Automatic Code Validation and Refinement in Natural Language to Shell Translation*. [Online]
Available at: <https://arxiv.org/abs/2404.17195>
[Accessed 2 mar 2026].

Lewis, P., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, Volume 33, p. 9459–9474.

Ylonen, T. L. C., 2006. *The Secure Shell (SSH) Protocol Architecture*. s.l.:s.n.

The Linux Documentation Project, 2026. *Cron Documentation*. [Online]
Available at: <https://tldp.org/>
[Accessed 12 mar 2026].